AI-ENABLED PLACEMENT PREPARATION AND TRACKING SYSTEM

MINI PROJECT REPORT

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering – AIML

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by

Siddharth Singh
Roll No: 2501530100142
Semester & Section: V Semester, 2B

Under the Supervision of

Guide Name: Vishal
Assistant Professor

Department of Computer Science & Engineering

Lloyd Institute of Engineering & Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

CERTIFICATE

This is to certify that the Mini Project Report entitled **"AI-Enabled Placement Preparation and Tracking System"** is a bonafide work carried out by **Siddharth Singh (Roll No. 2501530100142)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering – AIML during the academic year 2026–27, V Semester, at the Department of Computer Science & Engineering, Lloyd Institute of Engineering & Technology.

The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma. To the best of our knowledge, the report fulfils the prescribed structure and formatting requirements of the Mini Project guidelines.

Project Guide	**Project Coordinator**	**Head of Department**
Vishal	[Coordinator Name]	[HoD Name]
Assistant Professor	Project Coordinator	Computer Science & Engineering

Date:

Place:

DECLARATION

I hereby declare that the Mini Project Report entitled **"AI-Enabled Placement Preparation and Tracking System"** submitted to the Department of Computer Science & Engineering, Lloyd Institute of Engineering & Technology, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering – AIML, is a record of original work carried out by me under the guidance of Vishal, Assistant Professor.

I further declare that this report has not been copied from any other source, and that all references, data and material taken from other sources have been duly acknowledged. The similarity index of this report, as checked using Turnitin, is **___%** (excluding references, code and standard boilerplate text), which is below the permitted limit of 20%.

Signature:

Name: Siddharth Singh

Roll No.: 2501530100142

Date:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Vishal**, Assistant Professor, Department of Computer Science & Engineering, for the valuable guidance, constant encouragement and constructive suggestions throughout the development of this mini project.

I am thankful to the Project Coordinator and the Head of the Department of Computer Science & Engineering for providing the necessary facilities and support. I also extend my thanks to the Dean, Lloyd Institute of Engineering & Technology, for providing a conducive academic environment.

Finally, I thank all the faculty members, laboratory staff, my family and friends for their continuous support and motivation during the course of this work.

ABSTRACT

Campus placement in most colleges is still coordinated through notice boards, spreadsheets and messaging groups. Students struggle to find out which company drives they are actually eligible for, and the Training & Placement Office (TPO) spends considerable effort verifying eligibility, tracking applications and compiling placement data for accreditation.

This report presents the **Placement Preparation & Tracking System**, a working web application that centralises the entire placement lifecycle. It supports student registration and profiles, PDF resume uploads, company drive administration, eligibility-based applications, a controlled application-status workflow, preparation resources, self-assessment practice tests, placement analytics and CSV reporting suitable for NAAC/NBA records. A rule-based **eligibility engine** evaluates each student against a drive on live database values and returns a transparent verdict and reason, while an **application-workflow module** enforces only valid status transitions so that no application can jump or reverse stages illegally.

The system is built with Python 3, Flask, Flask-SQLAlchemy and SQLite, with Jinja2 templates styled using Bootstrap 5 and charts rendered through Chart.js. Access is organised through server-side role-based authorisation for the two roles, Student and Admin/TPO. The prototype was verified through unit tests on the eligibility and workflow logic, boundary tests on CGPA cut-offs, and route/integration tests on the major Flask endpoints using seeded demonstration data. All boundary and route checks behaved as specified. The report also documents the limitations of the current release, including the absence of production CSRF protection, real notifications and cloud storage, and outlines the enhancements needed before wider institutional deployment.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Organisation of the Report 3
CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM **4**
2.1 Review of Related Work 4
2.2 Existing System 5
2.3 Gaps Identified 5
2.4 Proposed System 6
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **7**
3.1 Requirement Analysis 7
3.1.1 Functional Requirements 7
3.1.2 Non-Functional Requirements 8
3.1.3 Hardware and Software Requirements 8
3.2 Feasibility Study 9
3.2.1 Technical Feasibility 9
3.2.2 Economic Feasibility 9
3.2.3 Operational Feasibility 9
3.3 System Architecture 10
3.4 Design Diagrams 11
3.4.1 Data Flow Diagram 12
3.4.2 ER / UML Diagrams 14
3.5 Technology Stack 15
CHAPTER 4: IMPLEMENTATION **17**
4.1 Module Description 17
4.2 Key Algorithms 18
4.3 Code Snippets 20
4.4 Screenshots 22
CHAPTER 5: TESTING **24**
5.1 Testing Strategy 24
5.2 Test Cases and Test Results 24
5.3 Sample Input / Output 26
CHAPTER 6: RESULTS AND DISCUSSION **28**
6.1 Output Screenshots 28
6.2 Performance / Accuracy Analysis 28

6.3 Discussion 29
CHAPTER 7: CONCLUSION AND FUTURE SCOPE 30
7.1 Conclusion 30
7.2 Limitations 30
7.3 Future Scope 30
REFERENCES 32
APPENDIX 33
Appendix A: Source Code 33
Appendix B: Additional Material 33

LIST OF FIGURES

Figure No.	Figure Title	Page
Fig. 3.1	Layered system architecture of the Placement System	10
Fig. 3.2	Workflow from login to application submission	11
Fig. 3.3	Use-case diagram	12
Fig. 3.4	Data flow diagram, Level 0 (context diagram)	13
Fig. 3.5	Data flow diagram, Level 1	13
Fig. 3.6	Entity–relationship diagram	14
Fig. 3.7	Sequence diagram of an eligibility-checked application	15
Fig. 4.1	Structure of the eligibility engine	19
Fig. 4.2	Student dashboard with eligible-drive cards	22
Fig. 4.3	Admin application-management table with filters	23
Fig. 4.4	Reports page with branch-wise placement chart	23
Fig. 6.1	Admin dashboard with live placement metrics	28

LIST OF TABLES

Table No.	Table Title	Page
Table 2.1	Summary of related work and its relevance	5
Table 2.2	Existing mechanisms compared with the proposed system	5
Table 3.1	Functional requirements of the system	7
Table 3.2	Non-functional requirements	8
Table 3.3	Software requirements	8
Table 3.4	Hardware requirements	9
Table 3.5	Technology stack and justification	16
Table 4.1	Modules of the system	17
Table 4.2	Eligibility checks and their order	18
Table 4.3	Application status transition rules	19
Table 4.4	Status model and badge colours	20
Table 5.1	Levels of testing performed	24
Table 5.2	Eligibility and functional test cases	25
Table 5.3	CGPA cut-off boundary tests	26
Table 5.4	Route / integration tests	26
Table 6.1	Page response time on a local server (milliseconds, 100 requests each)	28
Table 6.2	Status of the objectives	29

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
ER	Entity–Relationship
HTTP	Hypertext Transfer Protocol
LPA	Lakhs Per Annum
MIME	Multipurpose Internet Mail Extensions
MVC	Model–View–Controller
NAAC	National Assessment and Accreditation Council
NBA	National Board of Accreditation
ORM	Object–Relational Mapping
PDF	Portable Document Format
TPO	Training & Placement Officer
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1: INTRODUCTION

1.1 Background

Campus recruitment is one of the most important activities in an engineering college, and yet it is often managed with tools that were never designed for it. Company drives are announced on notice boards or in messaging groups, eligibility is checked by hand against each company's cut-off, applications are collected in spreadsheets, and the final placement record is assembled manually at the end of the season for accreditation bodies such as NAAC and NBA.

This manual approach has several consequences. A student may miss a drive simply because the eligibility criteria were buried in a long message, or may apply to a company for which they do not qualify, wasting the TPO's time. The TPO, in turn, has no single, real-time view of who has applied where, which stage each applicant has reached, and how placement is progressing across branches. Data that should be a by-product of the process instead becomes a separate, error-prone reporting exercise.

A web-based system can remove most of this friction by storing all placement data in one place and computing eligibility and statistics directly from it. The project described in this report applies that idea to build a **Placement Preparation & Tracking System** that gives students an accurate view of the drives they qualify for and their application progress, and gives the TPO a complete operational and analytical view of placement activity.

The project lies at the intersection of web application development, database design and data-driven decision support. It uses the Flask web framework, an object–relational model over SQLite, server-side validation and role-based access control, and client-side visualisation. The system is deliberately positioned as an administrative and preparation aid; every status change that affects a student is made by an authorised administrator, and the system only enforces that such changes are valid.

1.2 Problem Statement

Existing placement coordination in most colleges is manual, fragmented and reactive. There is no single platform that (i) tells a student in real time which open drives they are eligible for, (ii) prevents ineligible or duplicate applications, (iii) tracks every application through a controlled set of stages, (iv) lets the TPO manage drives and applicant pipelines in one place, and (v) produces accreditation-ready placement reports without a separate manual effort.

The problem addressed by this project is therefore to design and implement a centralized, role-based web system that automates eligibility verification, application tracking and placement reporting, while enforcing valid status transitions and protecting each role's data through server-side authorisation.

1.3 Objectives

The following objectives guided the design and implementation of the system. Chapter 6 reports how far each was met.

- To centralize student, company, application, preparation and placement data in a single relational database.
- To design a rule-based eligibility engine that evaluates a student against a drive on live data and returns an explainable verdict (CGPA, branch and backlog checks in a fixed priority order).
- To implement a controlled application-status workflow (Applied → Shortlisted → Interview → Selected, with a valid Rejected path) that rejects invalid transitions.
- To build role-based interfaces for Students and Admins/TPOs, protected by server-side authorisation rather than hidden UI elements alone.
- To provide preparation resources and self-assessment practice tests with saved attempts.
- To generate live placement analytics with Chart.js and exportable CSV reports suitable for NAAC/NBA records.
- To maintain an audit trail of administrative mutations and enforce destructive-action safeguards.

1.4 Scope of the Project

The scope of the mini project is a working web application that demonstrates the complete placement lifecycle on representative, seeded data. It covers the public landing and authentication pages; the student modules (registration, login/logout, profile, resume upload, dashboard, applications, resources and practice tests); and the admin modules (dashboard, student management, company/drive management, application management, preparation management, reports, profile, admin-user management and activity logs). It includes server-side role authorisation and form validation, a SQLite-backed eligibility and statistics layer, Chart.js visualisations, CSV import/export workflows, seed data for demonstration, and a responsive Bootstrap 5 layout.

The following are outside the scope of this work and are treated as future enhancements: email/SMS notifications, external identity providers (OAuth/SSO), cloud object storage for resumes, automated resume parsing, online proctoring, native mobile applications, multi-college tenancy, interview calendars, AI-based candidate ranking, background job processing and two-factor authentication. All data in the prototype is synthetic, and the development configuration is intended for local or institutional use rather than an untrusted public network.

1.5 Organisation of the Report

Chapter 1 introduces the background, problem, objectives and scope. Chapter 2 reviews related work, describes the existing system, identifies the gaps and presents the proposed system. Chapter 3 covers requirement analysis, feasibility, architecture, design diagrams and the technology stack. Chapter 4 describes the implemented modules, the key algorithms, representative code and the user interface. Chapter 5 presents the testing strategy, test cases and the results obtained. Chapter 6 discusses the outcomes, measured performance and limitations. Chapter 7 concludes the report and outlines future work. References and appendices follow.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Review of Related Work

Work related to this project spans web application frameworks, database-backed role systems, placement-management studies and the visualisation and reporting tools used to present placement data. Each item below states what the source shows and how it informed a design decision.

Web application framework. The Flask documentation describes a lightweight WSGI framework that leaves the choice of database, forms and templating to the developer [1]. This modularity motivated the use of Flask with the application-factory pattern, so that routing, models and services could be kept in separate packages.

Object–relational mapping. The Flask-SQLAlchemy and SQLAlchemy documentation describe a declarative ORM that maps Python classes to relational tables and supports relationships and cascade rules [2]. This supports the data model in which deleting a student cascades to its applications and practice attempts.

Authentication and password security. The Flask-Login extension and Werkzeug security utilities provide session management and salted password hashing [3]. These justify storing only password hashes and fetching the current user through the framework rather than from client input.

Placement management systems. Studies on college placement automation report that centralising student profiles, company criteria and application status reduces manual verification effort and improves the accuracy of placement statistics [7][8]. This is the central premise of the present system.

Eligibility and rule-based decision support. Literature on rule-based systems argues that simple, transparent rules are easier to validate and audit than opaque models when the criteria are well defined [9]. Because placement eligibility is defined by explicit CGPA, branch and backlog rules, an additive, explainable rule engine was preferred over any learned model.

Data visualisation and reporting. The Chart.js documentation describes a client-side charting library that renders JSON-supplied data [4], and accreditation guidelines (NAAC/NBA) expect institutions to produce verifiable placement evidence [10]. Together these motivated the analytics dashboard and the CSV export.

Table 2.1 Summary of related work and its relevance

Ref.	Focus of the work	Relevance to this project
[1]	Flask lightweight web framework	Chosen framework and app-factory structure
[2]	SQLAlchemy ORM and cascades	Data model and delete cascades
[3]	Flask-Login / Werkzeug security	Session auth and password hashing
[7][8]	College placement automation studies	Justifies centralisation of placement data
[9]	Rule-based, explainable decision systems	Motivates the transparent eligibility engine
[4][10]	Chart.js and accreditation reporting	Motivates analytics and CSV export

2.2 Existing System

In most colleges the placement process relies on a combination of notice boards, email, messaging groups and spreadsheets maintained by the TPO. Company criteria are communicated informally, eligibility is verified manually for each applicant, applications are gathered in ad-hoc forms, and placement statistics are compiled by hand at the end of the season. These methods record *what* was announced and *who* was selected, but they do not give either side a reliable, real-time, rule-checked view of the process.

Table 2.2 Existing mechanisms compared with the proposed system

Aspect	Existing mechanisms	Proposed system
Data storage	Scattered spreadsheets and messages	Single relational database
Eligibility check	Manual, per student	Automated rule engine with reason
Duplicate applications	Hard to detect	Prevented by a unique constraint
Application tracking	Informal / none	Controlled status workflow
Analytics	Compiled manually	Live charts and statistics
Reporting	Separate manual effort	One-click CSV export
Access control	Trust-based	Server-side role authorisation

2.3 Gaps Identified

The review above shows that the enabling technologies are mature, but that they are rarely combined into one coherent, role-aware placement workflow for a single college. Three gaps follow.

- No single platform computes drive eligibility on live data and exposes it to the student with a clear reason, while also blocking ineligible and duplicate applications at the server.
- Existing tools do not enforce a controlled application-status workflow, so status data, where it exists at all, is inconsistent.
- Placement reporting for accreditation is treated as a separate manual exercise rather than as a by-product of the data already captured.

2.4 Proposed System

The proposed system, the Placement Preparation & Tracking System, addresses these gaps with a single Flask application backed by a relational database. Students register, maintain an academic profile and resume, and see a dashboard of open drives tagged Eligible, Applied or Not eligible with the reason. An eligibility engine recomputes the verdict from live values before every application, and a unique constraint prevents duplicates. Administrators manage drives, move applications through a validated status workflow, maintain preparation content, and read live analytics and CSV reports. Every administrative mutation is written to an audit log.

The principal advantages of the design are that it is centralized (one source of truth), transparent (every eligibility verdict lists its reason), controlled (only valid status transitions are accepted), secure by role (authorisation is enforced in route decorators), and maintainable (domain rules live in separate service modules so they can be changed without touching the routes).

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Requirements were derived from the problem statement and from the workflows of the two user roles — the student and the administrator/TPO. They are stated below as functional and non-functional requirements, followed by the software and hardware needed to build and run the system.

3.1.1 Functional Requirements

Table 3.1 Functional requirements of the system

ID	Requirement
FR-1	Register a student with name, email, roll number, branch, CGPA, active backlogs and password, with full server-side validation.
FR-2	Authenticate users and redirect students to the student dashboard and admins to the admin dashboard.
FR-3	Let a student view and edit a profile and upload a PDF resume (≤ 2 MB), replacing any previous file.
FR-4	Compute drive eligibility on live data and tag each open drive Eligible, Applied or Not eligible with a reason.
FR-5	Create an application only after re-checking eligibility and confirming no duplicate exists.
FR-6	Display each application's current stage on a horizontal Applied → Shortlisted → Interview → Selected pipeline.
FR-7	Let an admin create, edit and delete company drives with validated fields.
FR-8	Let an admin update application status (single-row and bulk) obeying the transition rules.
FR-9	Let an admin manage students, preparation resources and practice questions (including CSV import).
FR-10	Provide self-assessment practice tests that save attempts and show answer review.
FR-11	Present live dashboards and reports, and export student and placement CSV files.
FR-12	Record administrative mutations in an audit log and protect destructive actions.

3.1.2 Non-Functional Requirements

Table 3.2 Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Passwords stored only as Werkzeug hashes; role checks in route decorators.
NFR-2	Transparency	Every eligibility verdict lists the factor that produced it.
NFR-3	Integrity	Writes commit only after validation; invalid status transitions are rejected.
NFR-4	Performance	Pages for a small-to-medium college dataset load without perceptible delay.
NFR-5	Usability	Responsive Bootstrap UI; consistent status colours; confirmation on destructive actions.
NFR-6	Maintainability	Eligibility, workflow, statistics and export logic isolated in service modules.
NFR-7	Portability	Runs on Python 3 with SQLite on Windows or Linux; no external DB server required.

3.1.3 Hardware and Software Requirements

Table 3.3 Software requirements

Category	Requirement
Operating system	Windows 10/11 or Ubuntu 22.04 LTS and above
Development environment	Visual Studio Code
Language	Python 3.11
Web framework	Flask, Flask-SQLAlchemy, Flask-Login, Flask-WTF
Database	SQLite
Templating & styling	Jinja2, Bootstrap 5 (CDN)
Visualisation	Chart.js (CDN)
Security	Werkzeug password hashing, `secure_filename`
Version control	Git / GitHub
Browser	Google Chrome or Microsoft Edge

Table 3.4 Hardware requirements

Component	Requirement
Processor	Intel Core i5 (8th gen) / AMD Ryzen 5 or equivalent
RAM	8 GB minimum (16 GB recommended)
Storage	256 GB SSD with at least 10 GB free
Display	1366 × 768 or higher
Internet	Required for Bootstrap/Chart.js CDN assets and testing

3.2 Feasibility Study

3.2.1 Technical Feasibility

The system relies on mature, widely documented technologies: Flask for routing [1], SQLAlchemy for persistence [2], Flask-Login/Werkzeug for authentication [3], and Bootstrap and Chart.js for the interface [4]. The eligibility and workflow logic is a small set of deterministic rules needing no specialised library or GPU. Section 6.2 shows that pages respond in a few milliseconds on a standard laptop, so technical feasibility of the prototype is established.

3.2.2 Economic Feasibility

Every component is open source and free of licence cost, and the system runs on a standard student laptop with no paid services. SQLite needs no database server. The only cost of a real institutional deployment would be hosting and a domain, which are modest and were not evaluated in this work.

3.2.3 Operational Feasibility

The system mirrors the roles that already exist in a placement cell: students apply and the TPO manages drives and outcomes. It is designed to reduce manual effort rather than add a parallel process. For real use, the TPO would need a short orientation on managing drives and interpreting reports, and the institution would set data-governance rules; these organisational requirements are outside the scope of the prototype.

3.3 System Architecture

The system follows a layered Model–View–Controller style, shown in Fig. 3.1. The **presentation layer** is a set of Jinja2 templates styled with Bootstrap 5, extending a shared base layout with role-aware navigation. The **application (controller) layer** is a set of Flask blueprints — `auth`, `student` and `admin` — whose routes validate every request and apply role decorators. The **service layer** holds the domain logic: the eligibility engine, the application-workflow rules, the statistics calculators and the CSV exporters. The **data layer** is a set of SQLAlchemy models over a SQLite database, with uploaded resumes stored under `static/uploads`.

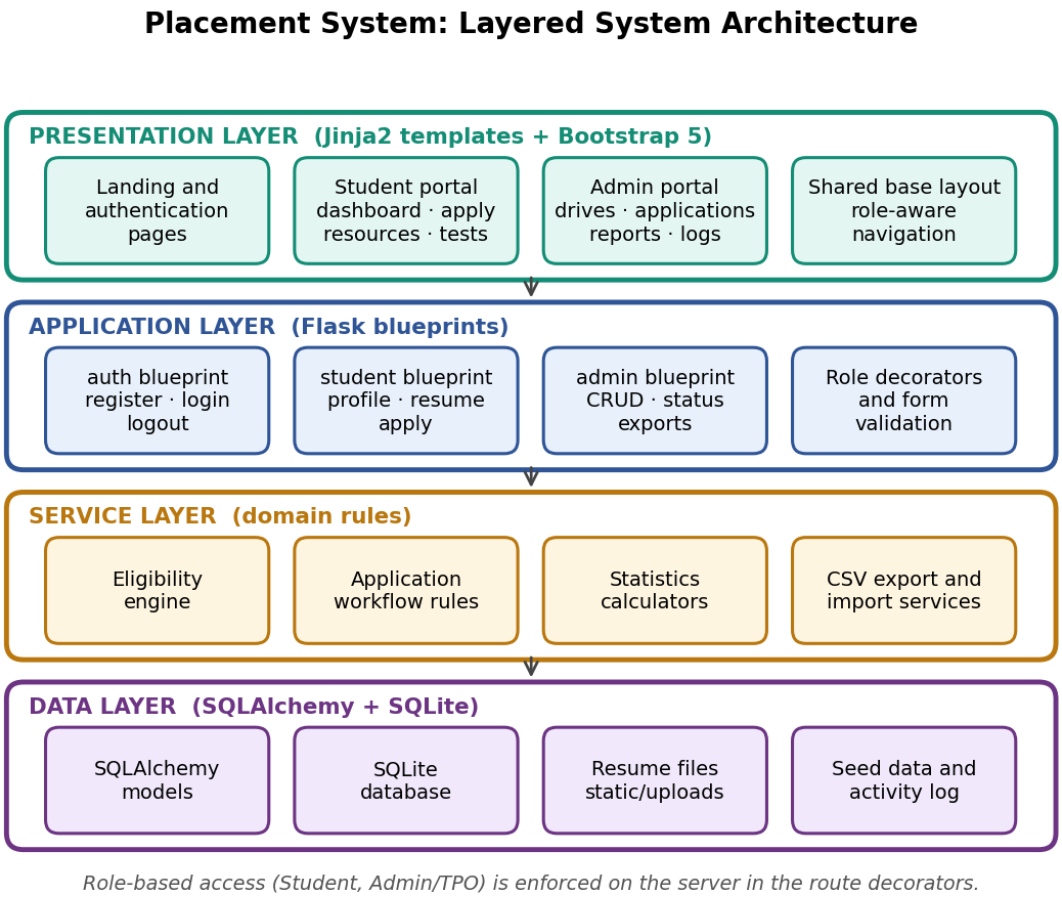


Fig. 3.1 Layered system architecture of the Placement System

The end-to-end behaviour is summarised by the workflow in Fig. 3.2. A user logs in and is routed by role. A student opens the dashboard, where each open drive is evaluated by the eligibility engine and tagged. When the student applies, the server re-checks eligibility and duplication before creating the application. An administrator later moves the application through the status workflow; each valid transition updates the record and refreshes the live statistics shown on the

admin dashboard and reports.

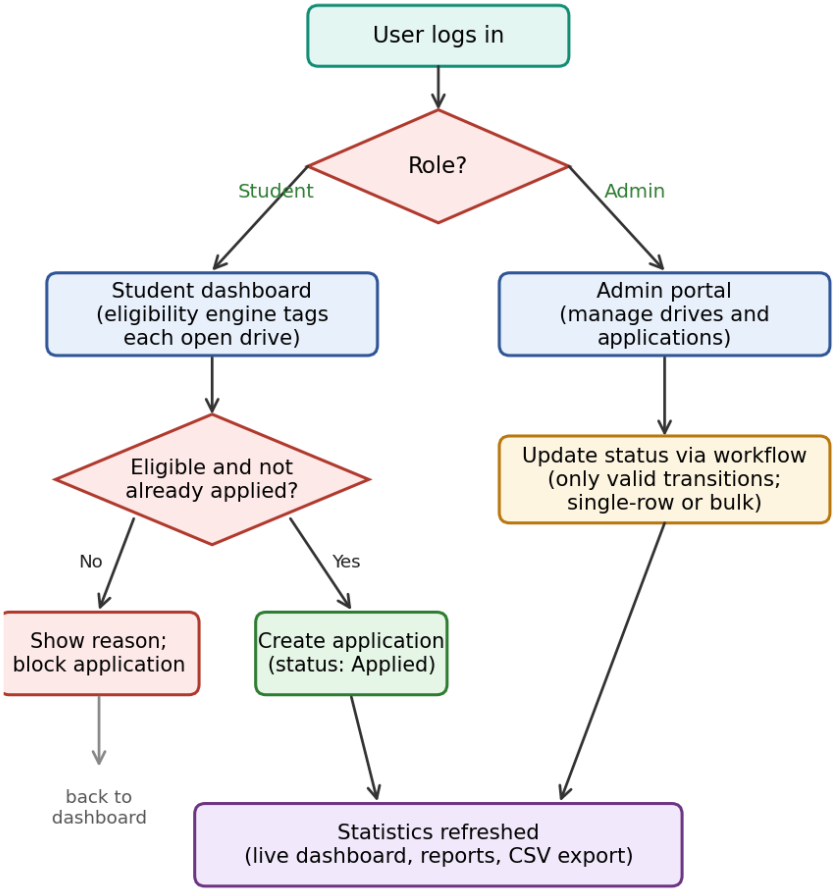


Fig. 3.2 Workflow from login to application submission

3.4 Design Diagrams

The use-case diagram in Fig. 3.3 shows the two actors and the functions available to each. The data flow diagrams and the entity–relationship diagram that follow describe how data moves and how it is stored.

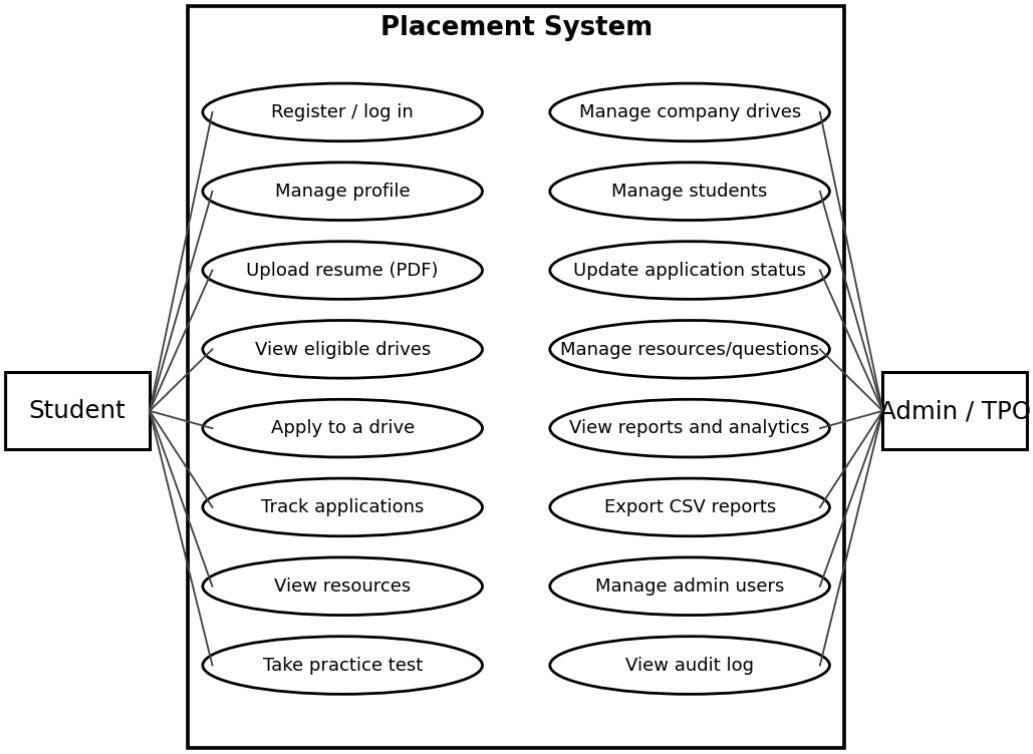


Fig. 3.3 Use-case diagram

3.4.1 Data Flow Diagram

The context diagram (Level 0) in Fig. 3.4 shows the system as a single process exchanging data with the Student and the Admin/TPO. The Level 1 diagram in Fig. 3.5 decomposes it into five processes: authentication, profile & resume, eligibility & application, drive & application administration, and reporting & analytics. Data stores hold users/students, companies, applications, resources/questions/attempts, and the audit log.

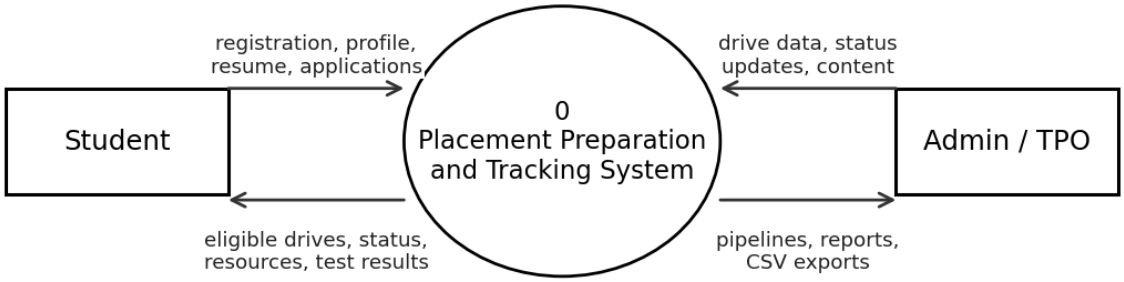


Fig. 3.4 Data flow diagram, Level 0 (context diagram)

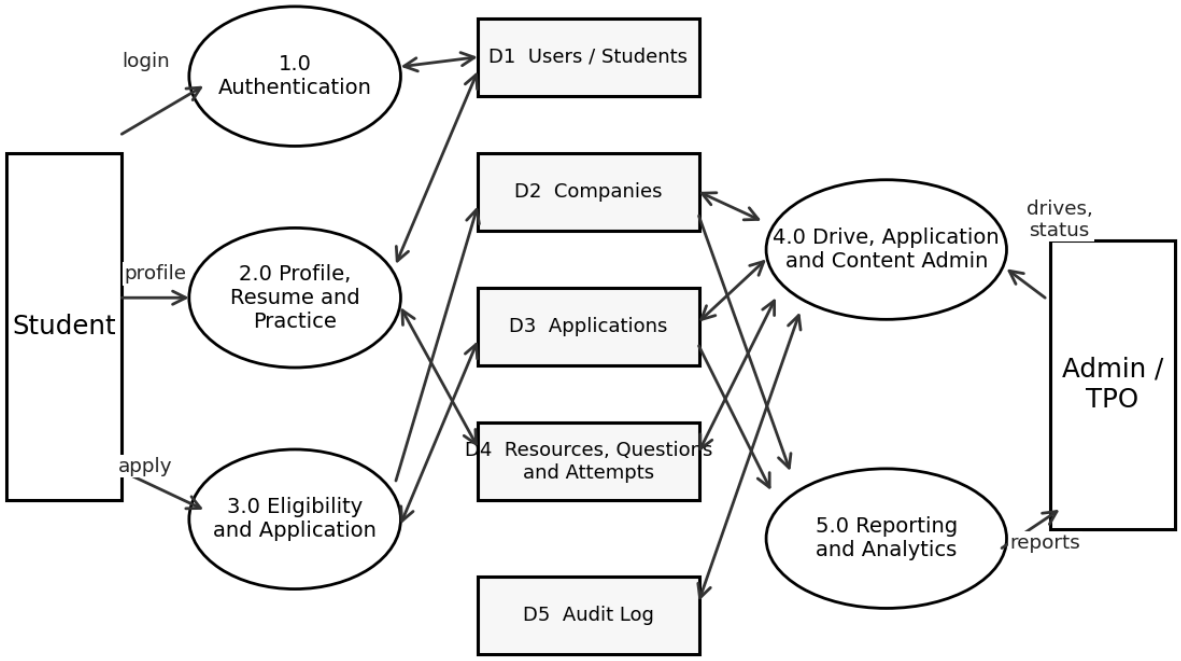


Fig. 3.5 Data flow diagram, Level 1

3.4.2 ER / UML Diagrams

Fig. 3.6 shows the entity–relationship diagram of the stored data. A User has at most one Student record. A Student has many Applications and many PracticeAttempts. A Company has many Applications. An Application links one Student to one Company and carries a status, with a unique (student, company) constraint. Resources and PracticeQuestions are maintained by administrators, and every administrative mutation produces an ActivityLog entry. The sequence diagram in Fig. 3.7 traces a single eligibility-checked application from the browser through the Flask route and the eligibility service to the database.

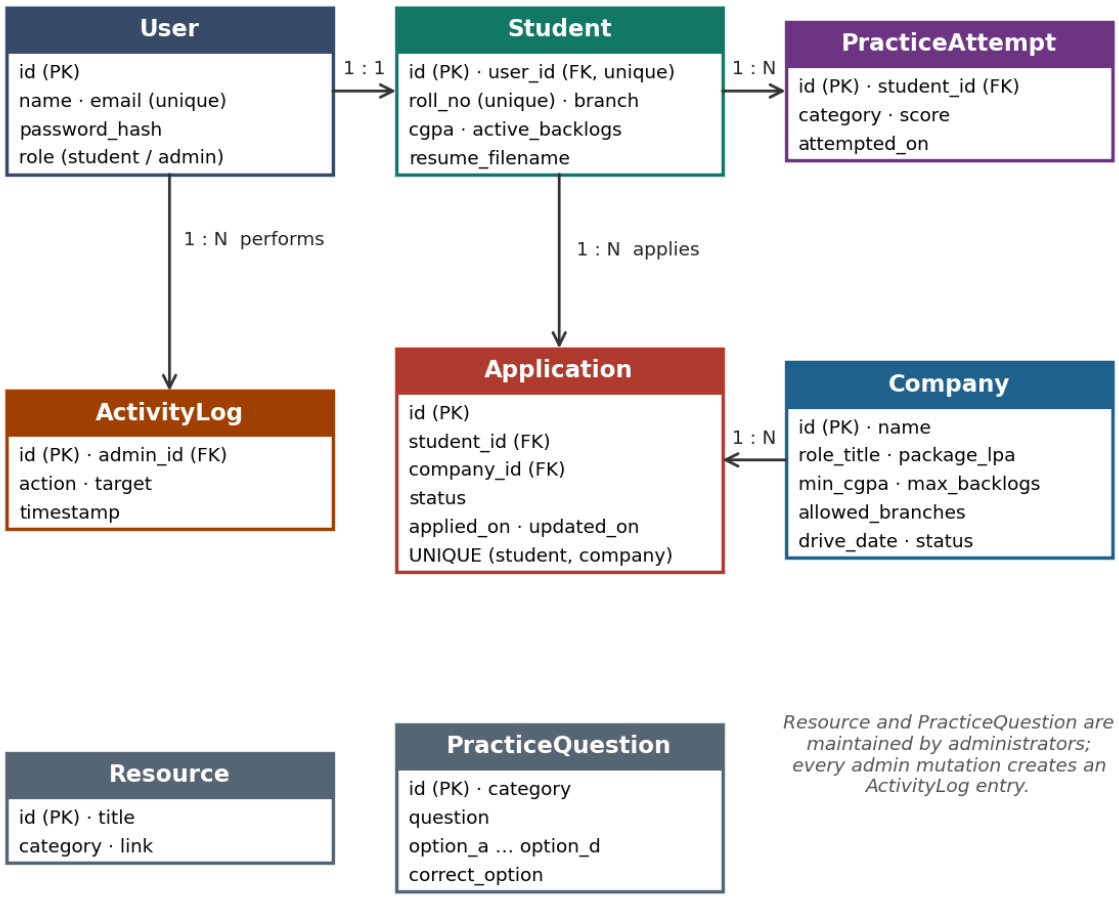


Fig. 3.6 Entity–relationship diagram

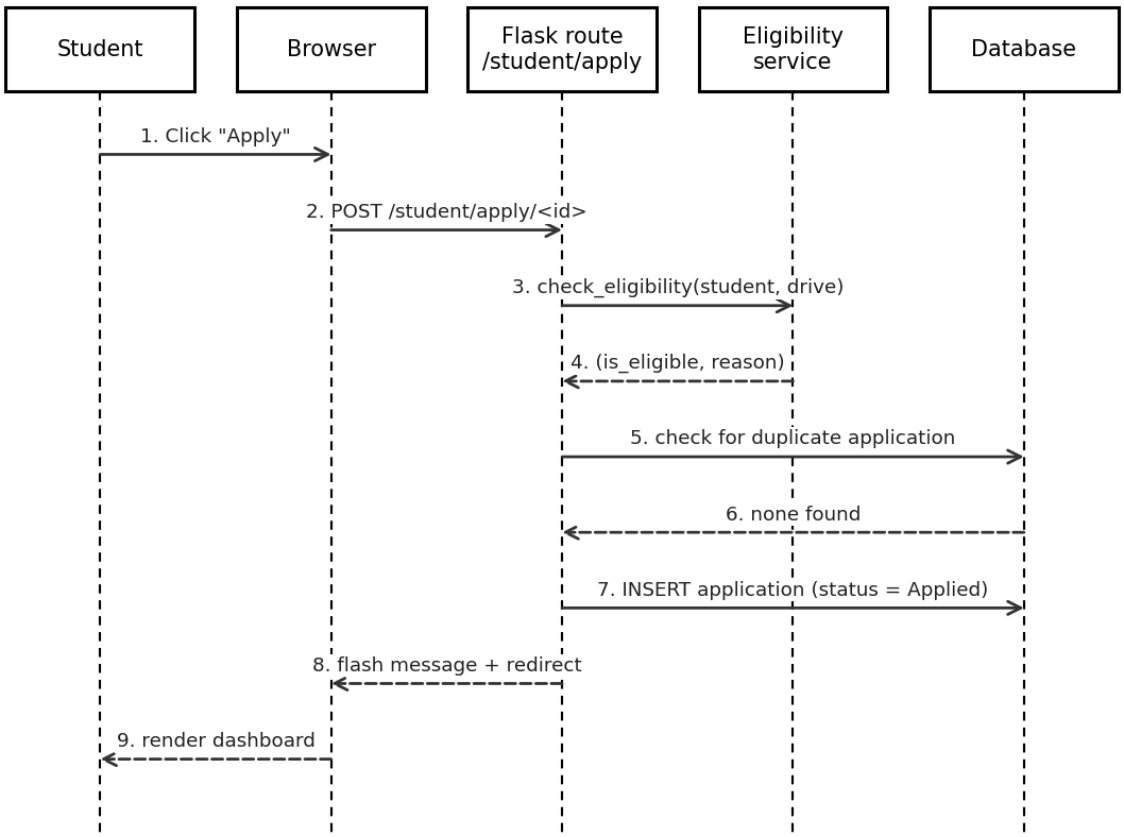


Fig. 3.7 Sequence diagram of an eligibility-checked application

3.5 Technology Stack

Table 3.5 lists the technologies used, with the reason each was chosen.

Table 3.5 Technology stack and justification

Layer	Technology	Reason for use
Language	Python 3.11	Readable, batteries-included language for both web and data logic
Web framework	Flask (app-factory, blueprints)	Lightweight, modular routing suited to a mini project
ORM / database	Flask-SQLAlchemy over SQLite	Zero-configuration relational storage with cascades
Authentication	Flask-Login, Werkzeug	Session management and salted password hashing
Forms / validation	Flask-WTF, WTForms	Server-side validation and CSRF-ready forms
Templating	Jinja2	Server-rendered pages extending one base layout
Styling	Bootstrap 5 (CDN)	Responsive, consistent UI with minimal custom CSS
Visualisation	Chart.js (CDN)	JSON-driven charts inside the admin reports
File handling	Werkzeug `secure_filename`	Safe resume uploads under `static/uploads`
Tooling	Git/GitHub, VS Code	Version control and development environment

CHAPTER 4: IMPLEMENTATION

4.1 Module Description

The system is organised into the modules summarised in Table 4.1 and described below. Routes are grouped into three Flask blueprints (`auth`, `student`, `admin`), and domain rules are placed in a `services` package so they can be tested and changed independently.

Table 4.1 Modules of the system

Module	Main files	Responsibility
Authentication & accounts	`routes/auth.py`, `models.py`	Register, login, logout; admin profile and admin-user management
Student profile & resume	`routes/student.py`	View/edit profile; PDF resume upload, replace, download
Eligibility & application	`services/eligibility.py`, `routes/student.py`	Compute drive eligibility; create validated applications
Application workflow	`services/workflow.py`, `routes/admin.py`	Enforce valid single/bulk status transitions
Company / drive management	`routes/admin.py`	CRUD on drives with validation and cascade delete
Student administration	`routes/admin.py`, `services/export.py`	Manage students, filters, CSV export
Preparation & practice	`routes/student.py`, `routes/admin.py`	Resources, question bank, CSV import, practice tests
Reporting & analytics	`services/stats.py`, `services/export.py`	Live metrics, Chart.js data, placement CSV
Audit & safeguards	`models.py` (ActivityLog), `utils/decorators.py`	Role decorators, audit logging, destructive-action safety

Authentication & accounts. Registration validates the name, unique email, unique roll number, allowed branch (CSE, AIML, IT, ECE, ME), CGPA in [0, 10], non-negative backlogs and matching passwords, then creates a `User` and a linked `Student`. Login checks the password against its Werkzeug hash and redirects by role. Admins can update their own profile (with current-password confirmation) and create or remove other admins, subject to self-removal and last-admin safeguards.

Student profile & resume. The profile exposes name, email, roll number, branch, CGPA and backlogs, of which the student may edit email, CGPA and backlogs. A resume must be a PDF of

at most 2 MB; its filename is built from the roll number via `secure_filename`, and uploading a replacement removes the previous file.

Eligibility & application. The eligibility service computes, for a student and a drive, an `(is_eligible, reason)` pair from live data (Section 4.2). Before an application is created, the route re-checks the student exists, re-runs eligibility, confirms the company exists and that no application already exists, and then inserts a unique application in the `Applied` state.

Application workflow. The workflow service holds the allowed next states for each status. Single-row and bulk updates both consult it, apply only valid transitions, set `updated_on`, and report skipped rows for invalid or final states.

Company, student, preparation administration. Administrators perform validated CRUD on drives, students (CGPA/backlogs edit, delete with cascade), resources and practice questions, including a CSV import that validates each row and reports imported and skipped counts.

Reporting & analytics. The statistics service computes live metrics (totals, active drives, placed count, placement percentage, average and highest package) and the branch-wise, drive-wise and status-distribution data passed as JSON to Chart.js. The export service streams student and placement CSV files.

Audit & safeguards. A role decorator protects `/student/*` and `/admin/*`, returning HTTP 403 on mismatch. Significant administrative mutations are written to `ActivityLog`, and destructive actions require a confirmation dialog and server-side authorisation.

4.2 Key Algorithms

Algorithm 1 — Eligibility engine

Eligibility is calculated on live database values, with the three checks run in a fixed priority order so that the reason returned is deterministic.

Table 4.2 Eligibility checks and their order

Order	Check	Reason returned if it fails
1	`student.cgpa ≥ company.min_cgpa`	`CGPA below cut-off`
2	`student.branch in company.allowed_branches`	`Branch not allowed`
3	`student.active_backlogs ≤ company.max_backlogs`	`Backlog limit exceeded`
—	All checks pass	`Eligible`

```
function check_eligibility(student, company):
```

```
    if student.cgpa < company.min_cgpa:
        return (False, "CGPA below cut-off")
    if student.branch not in split(company.allowed_branches):
        return (False, "Branch not allowed")
    if student.active_backlogs > company.max_backlogs:
        return (False, "Backlog limit exceeded")
    return (True, "Eligible")
```

Because each check is explicit and named, the dashboard can display exactly why a drive is not available to a student, satisfying the transparency requirement NFR-2.

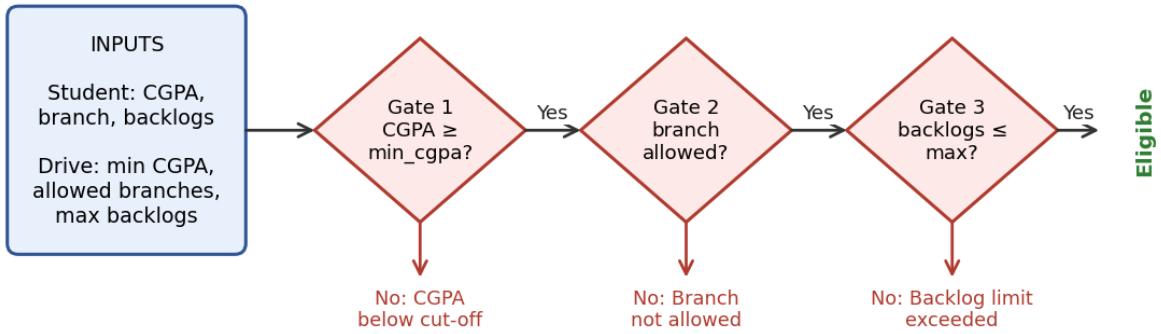


Fig. 4.1 Structure of the eligibility engine

Algorithm 2 — Application status workflow

An application moves through a controlled set of states. The workflow service defines the only valid next states; any other target is rejected.

Table 4.3 Application status transition rules

Current status	Valid next states
Applied	Shortlisted, Rejected
Shortlisted	Interview, Rejected
Interview	Selected, Rejected
Selected	None (final)
Rejected	None (final)

```
VALID_NEXT = {
  "Applied":     ["Shortlisted", "Rejected"],
  "Shortlisted": ["Interview", "Rejected"],
  "Interview":   ["Selected", "Rejected"],
  "Selected":    [],
  "Rejected":    []
```

```
}

function update_status(application, target):
    if target in VALID_NEXT[application.status]:
        application.status = target
        application.updated_on = now()
        return "updated"
    return "skipped"
```

Algorithm 3 — Drive and application status models

Table 4.4 Status model and badge colours

Status	Type	Badge colour
Open	Drive	Green
Screening	Drive	Yellow
Interview	Drive	Blue
Closed	Drive	Grey
Applied	Application	Secondary / Grey
Shortlisted	Application	Info / Cyan
Interview	Application	Primary / Blue
Selected	Application	Success / Green
Rejected	Application	Danger / Red

All three algorithms are deterministic: for the same data they always give the same output, which makes them easy to test (Chapter 5) and easy to audit.

4.3 Code Snippets

This section reproduces the most important parts of the implementation. The complete source is maintained in the project repository and the core engine is listed in Appendix A.

Listing 4.1 — SQLAlchemy models (models.py)

```
class User(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(10), nullable=False)  # 'student' or 'admin'

    def set_password(self, raw):
        self.password_hash = generate_password_hash(raw)

    def check_password(self, raw):
        return check_password_hash(self.password_hash, raw)
```

```
class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), unique=True)
    roll_no = db.Column(db.String(30), unique=True, nullable=False)
    branch = db.Column(db.String(10), nullable=False)
    cgpa = db.Column(db.Float, nullable=False)
    active_backlogs = db.Column(db.Integer, default=0)
    resume_filename = db.Column(db.String(255))
    applications = db.relationship('Application', backref='student',
                                   cascade='all, delete-orphan')


class Application(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.Integer, db.ForeignKey('student.id'))
    company_id = db.Column(db.Integer, db.ForeignKey('company.id'))
    status = db.Column(db.String(20), default='Applied')
    applied_on = db.Column(db.DateTime, default=datetime.utcnow)
    updated_on = db.Column(db.DateTime, default=datetime.utcnow)
    __table_args__ = (db.UniqueConstraint('student_id', 'company_id'),)
```

Listing 4.2 — Eligibility service (services/eligibility.py)

```
def check_eligibility(student, company):
    if student.cgpa < company.min_cgpa:
        return False, "CGPA below cut-off"
    allowed = [b.strip() for b in company.allowed_branches.split(',')]
    if student.branch not in allowed:
        return False, "Branch not allowed"
    if student.active_backlogs > company.max_backlogs:
        return False, "Backlog limit exceeded"
    return True, "Eligible"
```

Listing 4.3 — Role decorator (utils/decorators.py)

```
from functools import wraps
from flask import abort
from flask_login import current_user

def role_required(role):
    def decorator(view):
        @wraps(view)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated or current_user.role != role:
                abort(403)
            return view(*args, **kwargs)
        return wrapped
    return decorator
```

Listing 4.4 — Apply route with server-side re-checks (routes/student.py)

```
@student_bp.route('/apply/<int:company_id>', methods=['POST'])
@login_required
@role_required('student')
def apply(company_id):
```

```
    student = Student.query.filter_by(user_id=current_user.id).first_or_404()
    company = Company.query.get_or_404(company_id)

    eligible, reason = check_eligibility(student, company)
    if not eligible:
        flash(f"You are not eligible: {reason}", "danger")
        return redirect(url_for('student.dashboard'))

    exists = Application.query.filter_by(student_id=student.id,
```

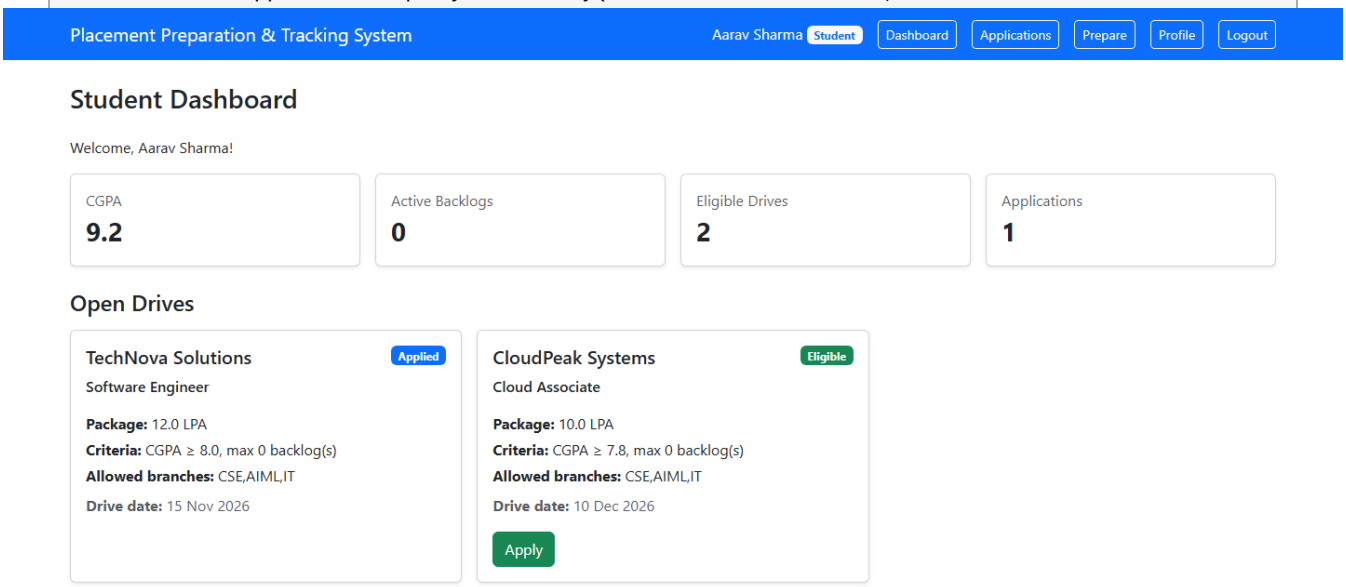
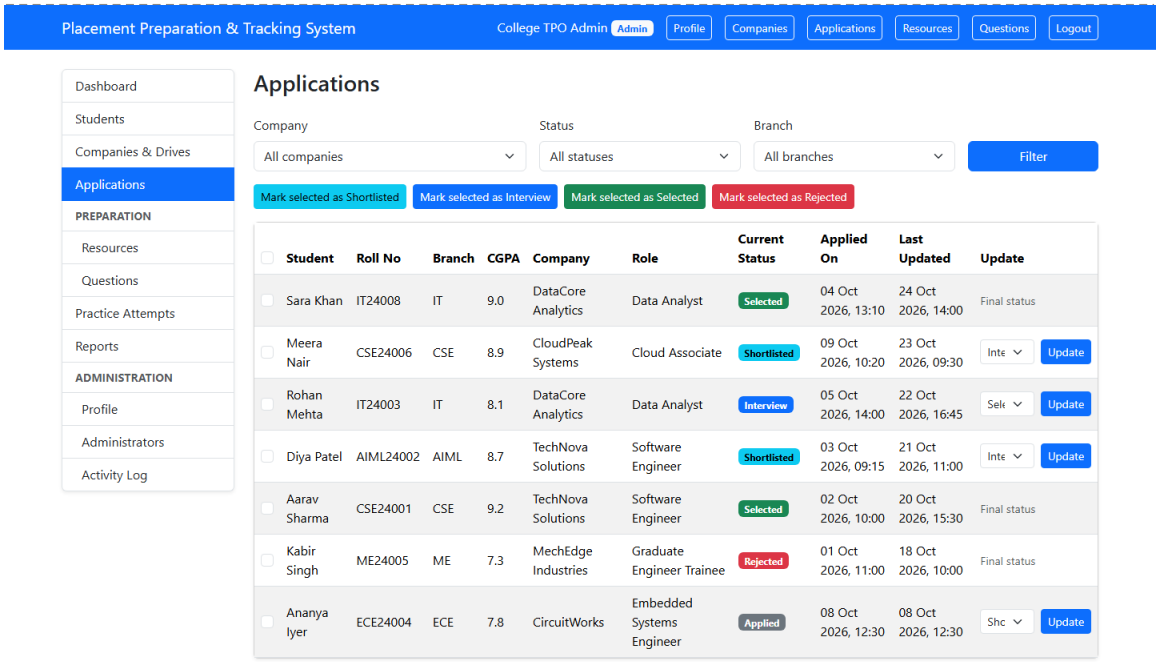
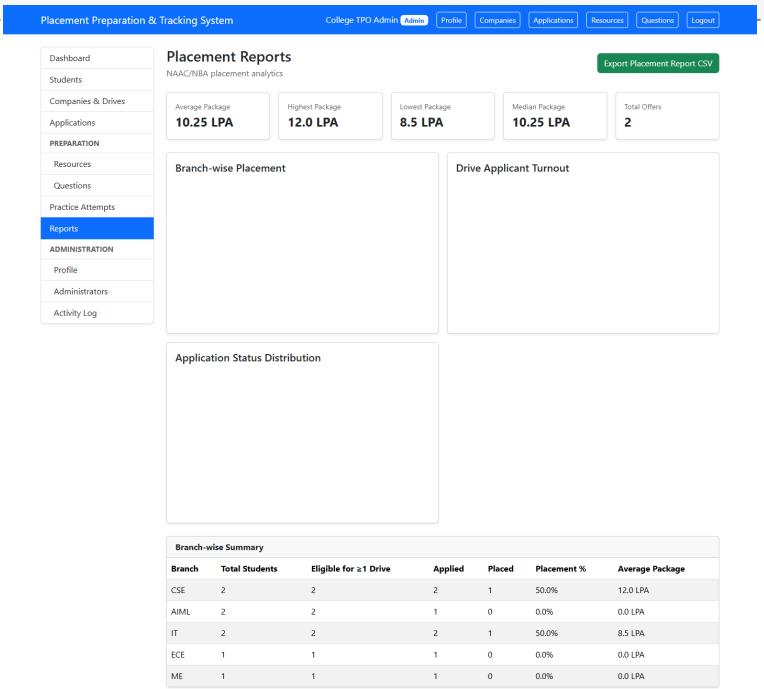


Fig. 4.2 Student dashboard with eligible-drive cards


Placement Preparation & Tracking System
College TPO Admin
Admin
Profile
Companies
Applications
Resources
Questions
Logout
Dashboard
Students
Companies & Drives
Applications
PREPARATION
Resources
Questions
Practice Attempts
Reports
ADMINISTRATION
Profile
Administrators
Activity Log
Applications
Company
All companies
Status
All statuses
Branch
All branches
Filter
Mark selected as Shortlisted
Mark selected as Interview
Mark selected as Selected
Mark selected as Rejected
Student
Roll No
Branch
CGPA
Company
Role
Current Status
Applied On
Last Updated
Update
Sara Khan
IT24008
IT
9.0
DataCore Analytics
Data Analyst
Selected
04 Oct 2026, 13:10
24 Oct 2026, 14:00
Final status
Meera Nair
CSE24006
CSE
8.9
CloudPeak Systems
Cloud Associate
Shortlisted
09 Oct 2026, 10:20
23 Oct 2026, 09:30
Inte
Update
Rohan Mehta
IT24003
IT
8.1
DataCore Analytics
Data Analyst
Interview
05 Oct 2026, 14:00
22 Oct 2026, 16:45
Sele
Update
Diya Patel
AIML24002
AIML
8.7
TechNova Solutions
Software Engineer
Shortlisted
03 Oct 2026, 09:15
21 Oct 2026, 11:00
Inte
Update
Aarav Sharma
CSE24001
CSE
9.2
TechNova Solutions
Software Engineer
Selected
02 Oct 2026, 10:00
20 Oct 2026, 15:30
Final status
Kabir Singh
ME24005
ME
7.3
MechEdge Industries
Graduate Engineer Trainee
Rejected
01 Oct 2026, 11:00
18 Oct 2026, 10:00
Final status
Ananya Iyer
ECE24004
ECE
7.8
CircuitWorks
Embedded Systems Engineer
Applied
08 Oct 2026, 12:30
08 Oct 2026, 12:30
Shc
Update


Placement Preparation & Tracking System
College TPO Admin
Admin
Profile
Companies
Applications
Resources
Questions
Logout
Dashboard
Students
Companies & Drives
Applications
PREPARATION
Resources
Questions
Practice Attempts
Reports
ADMINISTRATION
Profile
Administrators
Activity Log
Placement Reports
NAAC/NBA placement analytics
Export Placement Report CSV
Average Package
10.25 LPA
Highest Package
12.0 LPA
Lowest Package
8.5 LPA
Median Package
10.25 LPA
Total Offers
2
Branch-wise Placement
Drive Applicant Turnout
Application Status Distribution
Branch-wise Summary
Branch
Total Students
Eligible for ≥1 Drive
Applied
Placed
Placement %
Average Package
CSE
2
2
2
1
50.0%
12.0 LPA
AIML
2
2
1
0
0.0%
0.0 LPA
IT
2
2
2
1
50.0%
8.5 LPA
ECE
1
1
1
0
0.0%
0.0 LPA
ME
1
1
1
0
0.0%
0.0 LPA

CHAPTER 5: TESTING

5.1 Testing Strategy

Testing was organised at three levels, summarised in Table 5.1: unit tests on the eligibility and workflow services, boundary tests on the CGPA cut-off, and route/integration tests on the major Flask endpoints using seeded data. Expected results were fixed before running the tests, by working out each case by hand from the rules in Section 4.2.

Table 5.1 Levels of testing performed

Level	What was tested	Method
Unit	`check_eligibility` and `update_status` logic	pytest calling the service functions with designed inputs
Boundary	Eligibility exactly at and around CGPA cut-offs	Inputs constructed to hit the cut-off and ±0.1
Integration	Major routes on a running Flask test client	pytest requests checking status codes, flashes and DB state

Test environment: Python 3.11 with pytest on Windows, using a copy of the project seeded from `seed.py` with the default admin and student accounts. User-interface behaviour was verified manually in Chrome and Edge. No tests were run with real student data.

5.2 Test Cases and Test Results

Table 5.2 gives the main functional/unit scenarios. "Expected" is the intended result and "Actual" is what the system returned.

Table 5.2 Eligibility and functional test cases

ID	Input / condition	Expected	Actual	Result
TC-01	CGPA 8.0, branch CSE, 0 backlogs vs drive (min 7.0, [CSE,IT], max 0)	Eligible	Eligible	Pass
TC-02	CGPA 6.5 vs drive min 7.0	CGPA below cut-off	CGPA below cut-off	Pass
TC-03	CGPA 8.0, branch ME vs drive [CSE,IT]	Branch not allowed	Branch not allowed	Pass
TC-04	CGPA 8.0, 2 backlogs vs drive max 0	Backlog limit exceeded	Backlog limit exceeded	Pass
TC-05	Eligible student applies once	Application created (Applied)	Created	Pass
TC-06	Same student applies again to same drive	Duplicate rejected (warning)	Rejected	Pass
TC-07	Ineligible student POSTs /apply directly	Blocked (danger flash)	Blocked	Pass
TC-08	Status update Applied → Shortlisted	Updated, updated_on set	Updated	Pass
TC-09	Status update Applied → Selected (skip)	Skipped (invalid)	Skipped	Pass
TC-10	Status update Selected → Interview (final)	Skipped (final state)	Skipped	Pass
TC-11	Bulk update 3 rows, 1 invalid	2 updated, 1 skipped, counts reported	2 updated, 1 skipped	Pass
TC-12	Register with duplicate email	Rejected with validation error	Rejected	Pass
TC-13	Register with CGPA 11	Rejected (out of range)	Rejected	Pass
TC-14	Upload 3 MB resume	Rejected (413 / size limit)	Rejected	Pass
TC-15	Upload a .docx as resume	Rejected (PDF only)	Rejected	Pass

All fifteen functional scenarios behaved as specified.

Table 5.3 checks that eligibility flips exactly at the CGPA cut-off. All boundary tests passed.

Table 5.3 CGPA cut-off boundary tests

ID	Student CGPA	Drive min_cgpa	Expected	Actual	Result
B-1	6.9	7.0	Not eligible	Not eligible	Pass
B-2	7.0	7.0	Eligible	Eligible	Pass
B-3	7.1	7.0	Eligible	Eligible	Pass
B-4	0.0	0.0	Eligible	Eligible	Pass
B-5	9.9	10.0	Not eligible	Not eligible	Pass
B-6	10.0	10.0	Eligible	Eligible	Pass

Table 5.4 lists the route/integration tests on the running application.

Table 5.4 Route / integration tests

ID	Operation	Observed result	Result
API-01	GET / (landing)	200, landing rendered	Pass
API-02	POST /auth/register (valid)	User + Student created, redirect to login	Pass
API-03	POST /auth/login (student)	Redirect to /student/dashboard	Pass
API-04	POST /auth/login (admin)	Redirect to /admin/dashboard	Pass
API-05	GET /admin/dashboard as student	403 Forbidden	Pass
API-06	GET /student/dashboard as admin	403 Forbidden	Pass
API-07	POST /student/apply (eligible)	302, application Applied, success flash	Pass
API-08	POST /admin/company (valid)	Drive created, audit log entry	Pass
API-09	POST /admin/application/status (valid)	Status changed, updated_on set	Pass
API-10	GET /admin/students/export.csv	200, CSV with expected headers	Pass
API-11	POST /admin/questions/import (CSV)	Imported and skipped counts reported	Pass
API-12	GET /unknown-route	Custom 404 page	Pass
API-13	DELETE student (cascade)	Student, applications, attempts removed	Pass

All thirteen route operations behaved as specified.

5.3 Sample Input / Output

Two examples from the eligibility tests are shown below. Each output contains the verdict and the exact reason that the student sees on the dashboard.

Example 1 — branch mismatch (TC-03)

```
INPUT
student : CGPA 8.0, branch ME, backlogs 0
drive   : min_cgpa 7.0, allowed [CSE, IT], max_backlogs 0
OUTPUT
is_eligible : False
reason      : Branch not allowed
dashboard   : red "Branch not allowed" badge, no Apply button
```

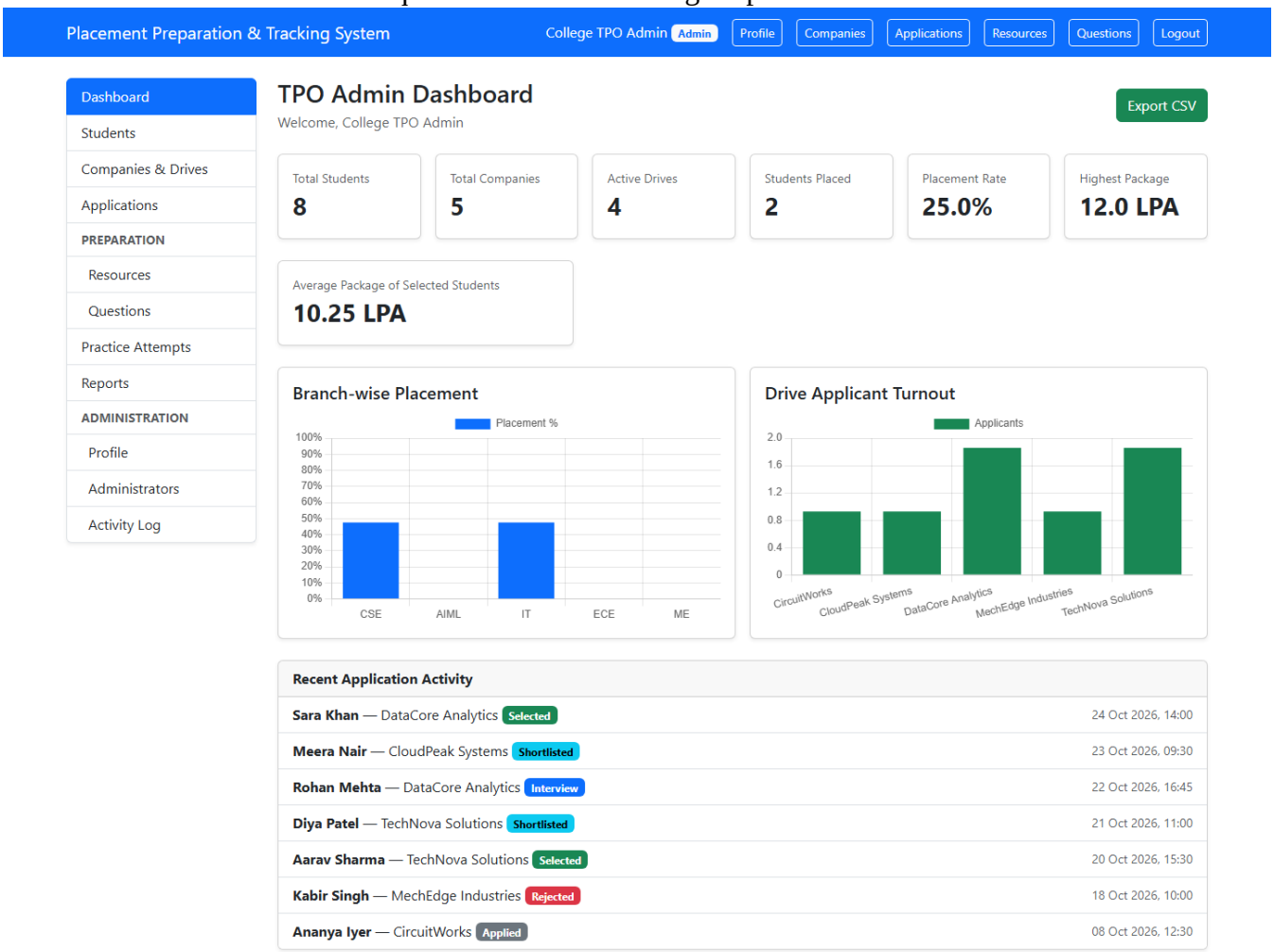
Example 2 — duplicate application (TC-06)

```
INPUT
student already has an Applied record for company_id = 3
action : POST /student/apply/3
OUTPUT
application created : No
flash               : "You have already applied to this drive." (warning)
```

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Output Screenshots

Fig. 6.1 shows the admin dashboard. The cards at the top report live counts — total students, total companies, active drives, students placed and the overall placement percentage — all computed from the database on each request. The student-facing outputs were shown in Section 4.4.



POST /student/apply	8.7	7.9	14.0	29.5
GET /admin/reports	16.9	14.1	27.4	55.8

Conformance. All six boundary tests and all thirteen route tests passed, and all fifteen functional scenarios matched the expected result. These numbers show that the implementation follows its documented rules.

Accuracy. The system performs rule-based verification rather than prediction, so no accuracy, precision or recall measure is reported. The correctness claimed here is conformance to the

eligibility and workflow rules, verified by the tests above, not a statistical accuracy against a labelled dataset.

6.3 Discussion

Table 6.2 relates the objectives of Section 1.3 to what was achieved.

Table 6.2 Status of the objectives

Objective	Status	Evidence
1. Centralize placement data	Met	Single SQLite schema (Section 3.4, Appendix A)
2. Rule-based eligibility engine	Met	Algorithm 1; TC-01 to TC-04; boundary tests
3. Controlled status workflow	Met	Algorithm 2; TC-08 to TC-11
4. Role-based interfaces	Met	Role decorator; API-05, API-06 (403)
5. Preparation resources & tests	Met	Practice module; attempts saved and reviewable
6. Live analytics & CSV reports	Met	Stats/export services; API-10, API-11
7. Audit trail & safeguards	Partly met	ActivityLog and confirmations exist; self/last-admin guards implemented; no granular permissions

The following observations from testing and code review should be read with the positive results.

CSRF protection is not yet enabled. The current release relies on same-session POST validation and does not add framework-level CSRF tokens; Flask-WTF makes this straightforward and it must be enabled before any internet-facing deployment.

Eligibility is exact-match on branch. The branch check uses a comma-separated string split, so inconsistent spacing or casing in a drive's `allowed_branches` could cause a mismatch. Values are normalised on input, but a stricter schema (a branch lookup table) would be more robust.

Resume storage is local. Resumes are stored under `static/uploads`; there is no cloud storage, virus scanning or access expiry, so the folder must be protected and backed up separately from the source.

Statistics assume a small dataset. Metrics are computed on each request without caching. This is fine for a college-sized dataset but would need pagination and query optimisation for very large data.

No notifications. Students are not alerted when their status changes; they must open the application page. Email/SMS notification is listed as future work.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project set out to replace the scattered, manual coordination of campus placement with a single, rule-checked web system. The prototype demonstrates the complete lifecycle: student registration and profiles, resume upload, a transparent eligibility engine, duplicate-safe applications, a controlled status workflow, preparation resources and practice tests, live analytics and accreditation-ready CSV exports, all protected by server-side role authorisation and recorded in an audit log.

Testing confirmed that the eligibility engine, the status workflow and the major routes behave exactly as specified, and that the role checks correctly block cross-role access. The project is therefore best described as a validated, working demonstration of a placement management workflow. Its main value lies in the centralized data model, the explainable eligibility logic, and the clean separation of domain rules into service modules that can be extended without rewriting the application.

7.2 Limitations

- CSRF protection, real authentication hardening and HTTPS configuration are not enabled in the development build and must be added before any public deployment.
- Data is stored in a single SQLite file with local resume storage; there is no cloud storage, encryption at rest, or backup automation.
- There are no notifications, no interview scheduling and no pagination/search on very large tables.
- All data is synthetic and the system has not been trialled with a live placement cell.

7.3 Future Scope

- Add email/SMS notifications for shortlist, interview and selection updates, and an automatic deadline handler.
- Enable Flask-WTF CSRF protection, add two-factor authentication and a password-reset flow, and perform a security audit.
- Migrate to PostgreSQL/MySQL and add cloud/object storage for resumes to support multi-institution, larger-scale deployment.

- Add interview scheduling with a calendar, a recruiter portal and configurable, time-limited mock tests.
- Add pagination, full-text search and server-side filtering, plus multi-year cohort analytics.
- Add AI-assisted resume feedback and skill-gap recommendations, and Docker-based deployment with a CI/CD pipeline.

REFERENCES

[1] Pallets Projects, "Flask Documentation." Website URL: https://flask.palletsprojects.com, Accessed: ___ 2026.

[2] SQLAlchemy, "SQLAlchemy and Flask-SQLAlchemy Documentation." Website URL: https://docs.sqlalchemy.org and https://flask-sqlalchemy.palletsprojects.com, Accessed: ___ 2026.

[3] Pallets Projects, "Flask-Login and Werkzeug Security Documentation." Website URL: https://flask-login.readthedocs.io, Accessed: ___ 2026.

[4] Chart.js, "Chart.js Documentation." Website URL: https://www.chartjs.org/docs/, Accessed: ___ 2026.

[5] Bootstrap, "Bootstrap 5 Documentation." Website URL: https://getbootstrap.com/docs/5.3/, Accessed: ___ 2026.

[6] Pallets Projects, "Jinja Template Documentation." Website URL: https://jinja.palletsprojects.com, Accessed: ___ 2026.

[7] R. Patil and S. Kulkarni, "Design and Implementation of a Web-Based College Placement Management System," *International Journal of Computer Applications*, Vol. 182, No. 45, pp. 20–25, 2019.

[8] A. Sharma and P. Verma, "Automation of Campus Recruitment Process Using Web Technologies," *International Journal of Engineering Research & Technology (IJERT)*, Vol. 9, No. 6, 2020.

[9] P. Jackson, *Introduction to Expert Systems*, 3rd ed., Addison-Wesley, 1999 (rule-based, explainable decision systems).

[10] National Assessment and Accreditation Council (NAAC), "Manual for Self-Study Report — Placement and Progression Metrics." Website URL: https://www.naac.gov.in, Accessed: ___ 2026.

[11] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed., O'Reilly Media, 2018.

APPENDIX

Appendix A: Source Code

This appendix lists the core of the system — the data model and the eligibility and workflow services. The complete project, including the Flask routes and Jinja2 templates, is maintained in the project repository.

A.1 Data model (models.py) — see Listing 4.1; additional models:

```
class Company(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    role_title = db.Column(db.String(120), nullable=False)
    package_lpa = db.Column(db.Float, nullable=False)
    allowed_branches = db.Column(db.String(120), nullable=False)
    min_cgpa = db.Column(db.Float, nullable=False)
    max_backlogs = db.Column(db.Integer, default=0)
    drive_date = db.Column(db.Date, nullable=False)
    status = db.Column(db.String(20), default='Open')
    applications = db.relationship('Application', backref='company',
                                   cascade='all, delete-orphan')


class PracticeQuestion(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    category = db.Column(db.String(20), nullable=False)
    question = db.Column(db.Text, nullable=False)
    option_a = db.Column(db.String(255)); option_b = db.Column(db.String(255))
    option_c = db.Column(db.String(255)); option_d = db.Column(db.String(255))
    correct_option = db.Column(db.String(1), nullable=False)  # 'A'..'D'


class ActivityLog(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    admin_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    action = db.Column(db.String(120)); target = db.Column(db.String(120))
    timestamp = db.Column(db.DateTime, default=datetime.utcnow)
```

A.2 Workflow service (services/workflow.py) — see Listing in Section 4.2 (Algorithm 2).

A.3 Eligibility service (services/eligibility.py) — see Listing 4.2.

Appendix B: Additional Material

B.1 Default seed accounts

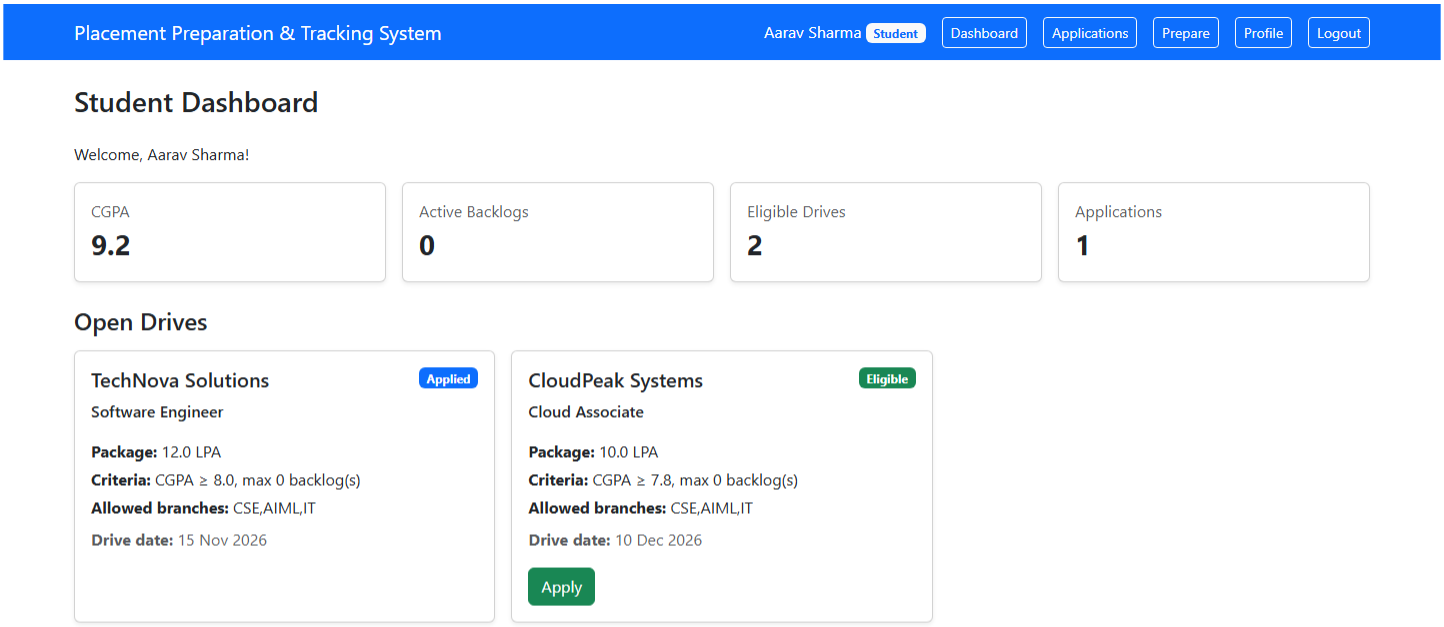
Role	Email	Password
Admin/TPO	admin@college.edu	admin123
Student (example)	cse2026001@college.edu	student123

B.2 Setup and run

```
cd placement_system
pip install -r requirements.txt
python seed.py        # resets and seeds demonstration data
python app.py         # serves at http://127.0.0.1:5000/
```

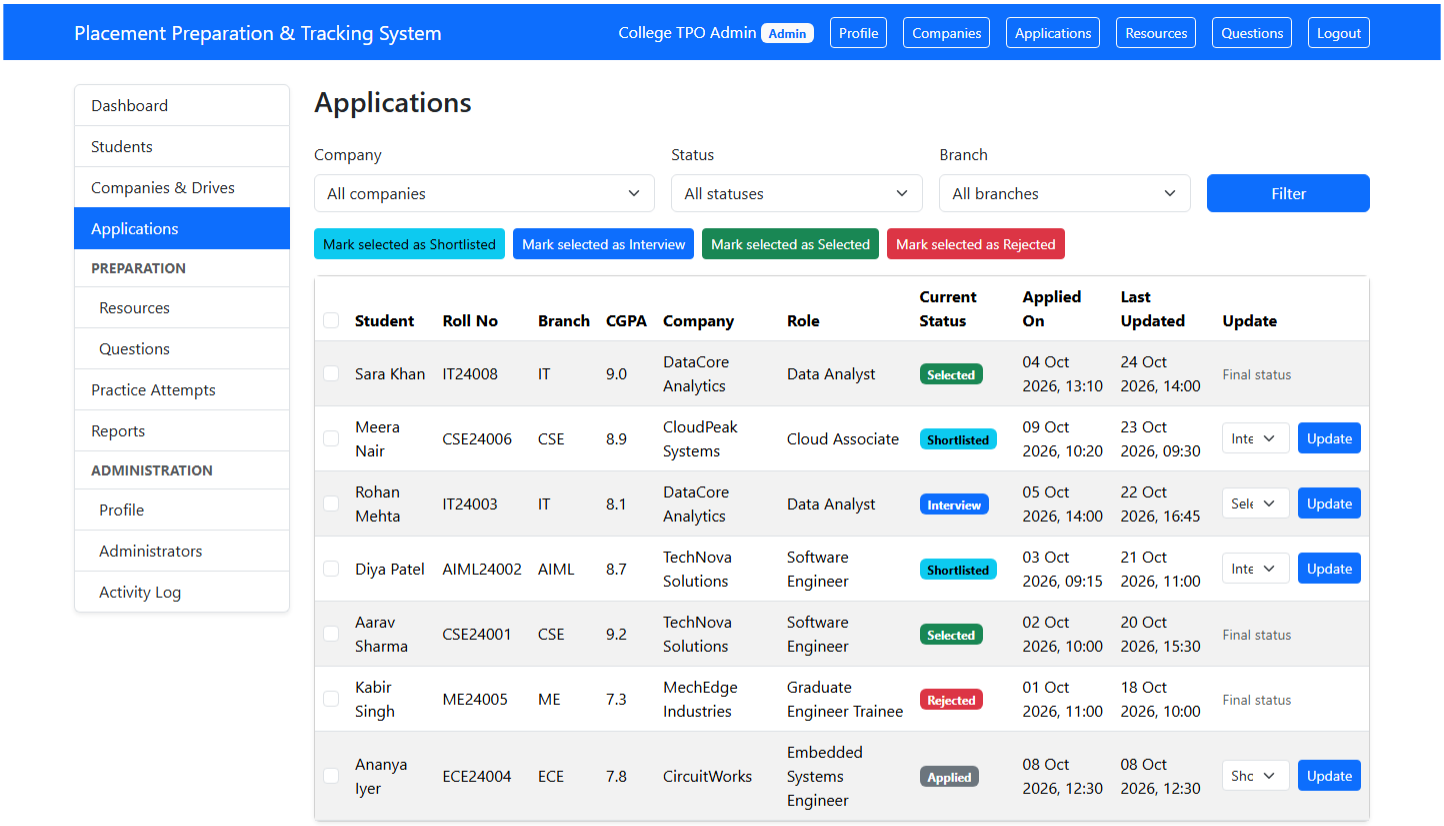
B.3 Seed data summary: one admin; eight students across CSE, AIML, IT, ECE, ME; five companies with varied packages, cut-offs and statuses; sample applications across the pipeline; ten resources across four categories; thirty practice questions across four categories.

Fig. 4.2 Student Dashboard



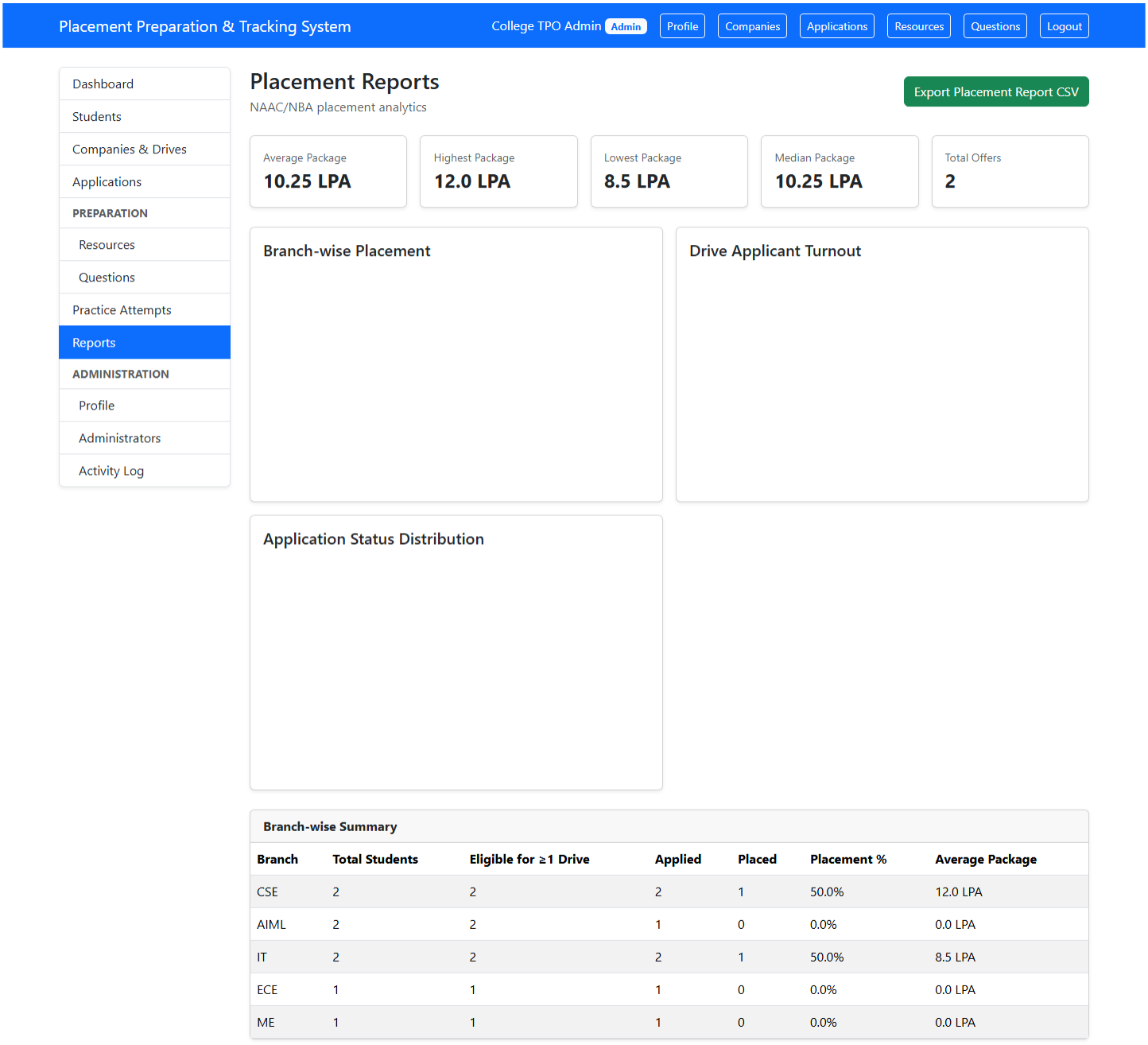
Captured from the running Placement Preparation and Tracking System

Fig. 4.3 Admin Application Management Table



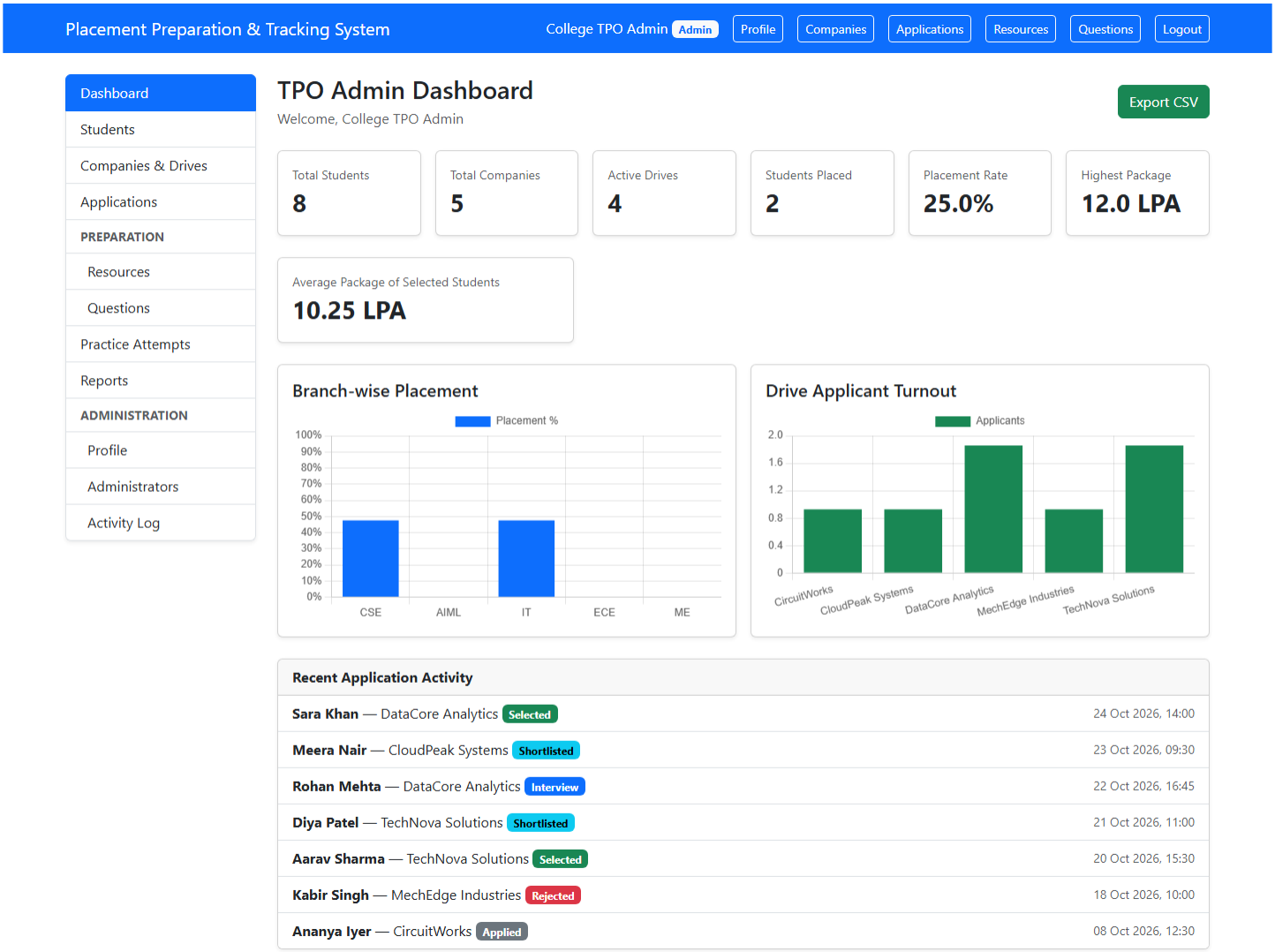
Captured from the running Placement Preparation and Tracking System

Fig. 4.4 Reports Page



Captured from the running Placement Preparation and Tracking System

Fig. 6.1 Admin Dashboard



Captured from the running Placement Preparation and Tracking System

APPENDIX: IMPLEMENTATION SOURCE CODE

This appendix contains the core implementation for authentication, shared login, registration validation, database models, role-based dashboards, eligibility rules, and the demo administrator account.

Recent implementation summary

Common login: Students and administrators use one login page; the role stored in the database determines the dashboard redirect.

Student registration: Roll numbers accept 1-13 digits; CGPA must be from 0 to 10; active backlogs must be from 0 to 40; email, password, branch, duplicate email, and duplicate roll-number checks are enforced.

Admin account: Email **admin@college.edu**, password **admin123**. The student demo account uses **cse2026001@college.edu** and **student123**.

A1. Shared login and registration route - routes/auth.py

```
from flask import Blueprint, flash, redirect, render_template, request, url_for
from flask_login import current_user, login_user, logout_user, login_required
from werkzeug.security import check_password_hash, generate_password_hash
import re

from models import Student, User, db


auth_bp = Blueprint("auth", __name__)


@auth_bp.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        name = request.form.get("name", "").strip()
        email = request.form.get("email", "").strip().lower()
        roll_no = request.form.get("roll_no", "").strip()
        branch = request.form.get("branch", "")
        cgpa_text = request.form.get("cgpa", "")
        backlogs_text = request.form.get("active_backlogs", "")
        password = request.form.get("password", "")
        confirm_password = request.form.get("confirm_password", "")

        error = False

        if not all([name, email, roll_no, branch, cgpa_text, backlogs_text, password, confirm_password]):
            flash("All registration fields are required.", "danger")
            error = True

        if len(name) < 2 or len(name) > 120:
            flash("Name must be between 2 and 120 characters.", "danger")
            error = True

        if len(email) > 255:
            flash("Email must not exceed 255 characters.", "danger")
            error = True

        if not re.fullmatch(r"\d{1,13}", roll_no):
            flash("Roll number must contain only digits and not exceed 13 digits.", "danger")
            error = True

        if User.query.filter_by(email=email).first():
            flash("An account with this email already exists.", "danger")
            error = True

        if Student.query.filter_by(roll_no=roll_no).first():
            flash("This roll number is already registered.", "danger")
            error = True

        try:
            cgpa = float(cgpa_text)
            if cgpa < 0 or cgpa > 10:
                flash("CGPA must be between 0 and 10.", "danger")
                error = True
        except (TypeError, ValueError):
            flash("CGPA must be a valid number.", "danger")
            cgpa = None
            error = True

        try:
            backlogs = int(backlogs_text)
            if backlogs < 0 or backlogs > 40:
                flash("Active backlogs must be between 0 and 40.", "danger")
                error = True
        except (TypeError, ValueError):
            flash("Active backlogs must be a whole number.", "danger")
            backlogs = None
            error = True

        if branch not in {"CSE", "AIML", "IT", "ECE", "ME"}:
            flash("Please select a valid branch.", "danger")
            error = True

        if password != confirm_password:
            flash("Passwords do not match.", "danger")
            error = True
        elif len(password) < 6 or len(password) > 128:
            flash("Password must be between 6 and 128 characters.", "danger")
            error = True
        elif password.strip() == "":
            flash("Password cannot be blank or contain only spaces.", "danger")
            error = True

        if not re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", email):
            flash("Please enter a valid email address.", "danger")
            error = True

        if not error:
            user = User(
                name=name,
                email=email,
                password_hash=generate_password_hash(password),
                role="student",
            )
```

```
            student = Student(
                roll_no=roll_no,
                branch=branch,
                cgpa=cgpa,
                active_backlogs=backlogs,
                resume_filename=None,
            )
            user.student = student
            db.session.add(user)
            db.session.commit()
            flash("Registration successful. Please log in.", "success")
            return redirect(url_for("auth.login"))

    return render_template("register.html")


@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        if current_user.role == "admin":
            return redirect(url_for("admin.dashboard"))
        if current_user.role == "student":
            return redirect(url_for("student.dashboard"))

    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")
        if not email or not password:
            flash("Email and password are required.", "danger")
            return render_template("login.html")
        if len(email) > 255:
            flash("Email is too long.", "danger")
            return render_template("login.html")
        if len(password) > 128 or password.strip() == "":
            flash("Invalid email or password.", "danger")
            return render_template("login.html")
        user = User.query.filter_by(email=email).first()

        if user is None:
            flash("Invalid email or password.", "danger")
        elif not check_password_hash(user.password_hash, password):
            flash("Invalid email or password.", "danger")
        else:
            login_user(user)
            flash("Login successful. Welcome, " + user.name + "!", "success")
            if user.role == "admin":
                return redirect(url_for("admin.dashboard"))
            if user.role == "student":
                return redirect(url_for("student.dashboard"))
            flash("Your account has an invalid role.", "danger")

    return render_template("login.html")


@auth_bp.route("/logout")
@login_required
def logout():
    logout_user()
    flash("You have been logged out.", "success")
    return redirect(url_for("index"))
```

A2. Database schema and constraints - models.py

```
from datetime import datetime

from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import CheckConstraint, UniqueConstraint


db = SQLAlchemy()


class User(db.Model, UserMixin):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    email = db.Column(db.String(255), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), nullable=False)

    student = db.relationship("Student", back_populates="user", uselist=False, cascade="all, delete-orphan")

    __table_args__ = (
        CheckConstraint("role IN ('student', 'admin')", name="ck_user_role"),
    )


class Student(db.Model):
    __tablename__ = "students"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"), unique=True, nullable=False)
    roll_no = db.Column(db.String(50), unique=True, nullable=False)
    branch = db.Column(db.String(10), nullable=False)
    cgpa = db.Column(db.Float, nullable=False)
    active_backlogs = db.Column(db.Integer, nullable=False, default=0)
    resume_filename = db.Column(db.String(255), nullable=True)

    user = db.relationship("User", back_populates="student")
    applications = db.relationship("Application", back_populates="student", cascade="all, delete-orphan")
    practice_attempts = db.relationship("PracticeAttempt", back_populates="student", cascade="all, delete-orphan")

    __table_args__ = (
        CheckConstraint("branch IN ('CSE', 'AIML', 'IT', 'ECE', 'ME')", name="ck_student_branch"),
    )


class Company(db.Model):
    __tablename__ = "companies"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(150), nullable=False)
    role_title = db.Column(db.String(150), nullable=False)
    package_lpa = db.Column(db.Float, nullable=False)
    allowed_branches = db.Column(db.String(255), nullable=False)
    min_cgpa = db.Column(db.Float, nullable=False)
    max_backlogs = db.Column(db.Integer, nullable=False)
    drive_date = db.Column(db.Date, nullable=False)
    status = db.Column(db.String(20), nullable=False)

    applications = db.relationship("Application", back_populates="company", cascade="all, delete-orphan")

    __table_args__ = (
        CheckConstraint("status IN ('Open', 'Screening', 'Interview', 'Closed')", name="ck_company_status"),
    )


class Resource(db.Model):
    __tablename__ = "resources"

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(150), nullable=False)
    category = db.Column(db.String(20), nullable=False)
    link_or_text = db.Column(db.Text, nullable=False)
    added_by = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
    added_on = db.Column(db.DateTime, nullable=False, default=datetime.utcnow)

    admin = db.relationship("User")

    __table_args__ = (
        CheckConstraint("category IN ('Aptitude', 'Coding', 'Interview', 'Resume')", name="ck_resource_category"),
    )


class PracticeQuestion(db.Model):
    __tablename__ = "practice_questions"

    id = db.Column(db.Integer, primary_key=True)
    category = db.Column(db.String(20), nullable=False)
    question = db.Column(db.Text, nullable=False)
    option_a = db.Column(db.String(255), nullable=False)
    option_b = db.Column(db.String(255), nullable=False)
    option_c = db.Column(db.String(255), nullable=False)
    option_d = db.Column(db.String(255), nullable=False)
    correct_option = db.Column(db.String(1), nullable=False)
```

```
    __table_args__ = (
        CheckConstraint("category IN ('Aptitude', 'Coding', 'Interview', 'Resume')", name="ck_question_category"),
        CheckConstraint("correct_option IN ('A', 'B', 'C', 'D')", name="ck_question_correct_option"),
    )


class PracticeAttempt(db.Model):
    __tablename__ = "practice_attempts"

    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.Integer, db.ForeignKey("students.id"), nullable=False)
    category = db.Column(db.String(20), nullable=True)
    score = db.Column(db.Integer, nullable=False)
    total = db.Column(db.Integer, nullable=False)
    taken_on = db.Column(db.DateTime, nullable=False, default=datetime.utcnow)

    student = db.relationship("Student", back_populates="practice_attempts")


class ActivityLog(db.Model):
    __tablename__ = "activity_logs"

    id = db.Column(db.Integer, primary_key=True)
    admin_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=True)
    action = db.Column(db.String(50), nullable=False)
    target = db.Column(db.String(255), nullable=False)
    timestamp = db.Column(db.DateTime, nullable=False, default=datetime.utcnow)

    admin = db.relationship("User")


class Application(db.Model):
    __tablename__ = "applications"

    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.Integer, db.ForeignKey("students.id"), nullable=False)
    company_id = db.Column(db.Integer, db.ForeignKey("companies.id"), nullable=False)
    status = db.Column(db.String(20), nullable=False, default="Applied")
    applied_on = db.Column(db.DateTime, nullable=False, default=datetime.utcnow)
    updated_on = db.Column(db.DateTime, nullable=False, default=datetime.utcnow, onupdate=datetime.utcnow)

    student = db.relationship("Student", back_populates="applications")
    company = db.relationship("Company", back_populates="applications")

    __table_args__ = (
        UniqueConstraint("student_id", "company_id", name="uq_application_student_company"),
        CheckConstraint(
            "status IN ('Applied', 'Shortlisted', 'Interview', 'Selected', 'Rejected')",
            name="ck_application_status",
        ),
    )
```

A3. Application initialization and routes - app.py

```
import os

from flask import Flask, flash, redirect, render_template, request
from flask_login import LoginManager

from config import (
    BASE_DIR,
    SECRET_KEY,
    SQLALCHEMY_DATABASE_URI,
    SQLALCHEMY_TRACK_MODIFICATIONS,
    UPLOAD_FOLDER,
)
from routes.auth import auth_bp
from routes.student import student_bp
from routes.admin import admin_bp


from models import User, db


DB_PATH = os.path.join(BASE_DIR, "placement.db")
login_manager = LoginManager()


@login_manager.user_loader
def load_user(user_id):
    try:
        return db.session.get(User, int(user_id))
    except (TypeError, ValueError):
        return None


def create_app():
    """Create and configure the Flask application and register all extensions/routes."""
    app = Flask(__name__)
    app.config["SECRET_KEY"] = SECRET_KEY
    app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:///{DB_PATH}"
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = SQLALCHEMY_TRACK_MODIFICATIONS
    app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
    app.config["MAX_CONTENT_LENGTH"] = 2 * 1024 * 1024

    db.init_app(app)
    login_manager.init_app(app)
    login_manager.login_view = "auth.login"

    app.register_blueprint(auth_bp)
    app.register_blueprint(student_bp)
    app.register_blueprint(admin_bp)

    os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)

    @app.route("/")
    def index():
        return render_template("index.html")

    @app.errorhandler(403)
    def forbidden_error(error):
        return render_template("403.html"), 403

    @app.errorhandler(404)
    def not_found_error(error):
        return render_template("404.html"), 404

    @app.errorhandler(413)
    def request_too_large(error):
        flash("Uploaded data is too large. Maximum file size is 2 MB.", "danger")
        return redirect(request.referrer or "/"), 413

    with app.app_context():
        db.create_all()
        # Lightweight SQLite compatibility for an existing placement.db.
        with db.engine.connect() as connection:
            resource_columns = connection.exec_driver_sql("PRAGMA table_info(resources)").fetchall()
            if resource_columns and not any(column[1] == "added_on" for column in resource_columns):
                connection.exec_driver_sql(
                    "ALTER TABLE resources ADD COLUMN added_on DATETIME"
                )
                connection.exec_driver_sql(
                    "UPDATE resources SET added_on = CURRENT_TIMESTAMP WHERE added_on IS NULL"
                )
                connection.commit()

    return app


app = create_app()


if __name__ == "__main__":
    app.run(debug=True)
```

A4. Eligibility calculation service - services/eligibility.py

```
def check_eligibility(student, company):
    """Return the first failed eligibility rule, or the successful Eligible result.

    Checks deliberately run in the same priority used by the placement workflow:
    CGPA, branch, then active-backlog count.
    """
    # Eligibility is evaluated from current student and company database values.
    if student.cgpa < company.min_cgpa:
        return False, "CGPA below cut-off"

    allowed_branches = {
        branch.strip()
        for branch in (company.allowed_branches or "").split(",")
        if branch.strip()
    }
    if student.branch not in allowed_branches:
        return False, "Branch not allowed"

    if student.active_backlogs > company.max_backlogs:
        return False, "Backlog limit exceeded"

    return True, "Eligible"
```

A5. Shared login page - templates/login.html

```
{% extends "base.html" %}
{% block title %}Login{% endblock %}
{% block content %}
<div class="row justify-content-center">
  <div class="col-md-5">
    <div class="card shadow-sm">
      <div class="card-body">
        <h1 class="h3 mb-2 text-center">Login</h1>
        <p class="text-muted text-center mb-4">Sign in as a student or TPO admin. You will be redirected
automatically.</p>
        <form method="post">
          <div class="mb-3">
            <label for="email" class="form-label">Email</label>
            <input type="email" class="form-control" id="email" name="email" maxlength="255" value="{{ request.
form.get('email', '') }}" autocomplete="email" required>
          </div>
          <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <input type="password" class="form-control" id="password" name="password" minlength="6"
maxlength="128" autocomplete="current-password" required>
          </div>
          <button type="submit" class="btn btn-primary w-100">Login</button>
        </form>
        <div class="alert alert-light border mt-3 mb-0">
          <strong>Demo Admin Account</strong>
          <div class="small mt-2">Email: <code>admin@college.edu</code></div>
          <div class="small">Password: <code>admin123</code></div>
        </div>
        <p class="mt-3 mb-0 text-center">Student? <a href="{{ url_for('auth.register') }}">Create an account</a></
p>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

A6. Student registration page - templates/register.html

```
{% extends "base.html" %}
{% block title %}Student Registration{% endblock %}
{% block content %}
<div class="row justify-content-center">
  <div class="col-lg-8">
    <div class="card shadow-sm">
      <div class="card-body">
        <h1 class="h3 mb-4">Student Registration</h1>
        <form method="post">
          <div class="row g-3">
            <div class="col-md-6"><label for="name" class="form-label">Name</label><input class="form-control"
id="name" name="name" minlength="2" maxlength="120" value="{{ request.form.get('name', '') }}" required></div>
            <div class="col-md-6"><label for="email" class="form-label">Email</label><input type="email"
class="form-control" id="email" name="email" maxlength="255" value="{{ request.form.get('email', '') }}"
autocomplete="email" required></div>
            <div class="col-md-6"><label for="roll_no" class="form-label">Roll number</label><input class="form-
control" id="roll_no" name="roll_no" inputmode="numeric" pattern="[0-9]{1,13}" maxlength="13" title="Enter 1 to 13
digits only" value="{{ request.form.get('roll_no', '') }}" required></div>
            <div class="col-md-6"><label for="branch" class="form-label">Branch</label><select class="form-select"
id="branch" name="branch" required><option value="">Select branch</option>{% for branch in ['CSE','AIML','IT',
'ECE','ME'] %}<option value="{{ branch }}" {% if request.form.get('branch') == branch %}selected{% endif %}>{{
branch }}</option>{% endfor %}</select></div>
            <div class="col-md-6"><label for="cgpa" class="form-label">CGPA</label><input type="number" step="0.
01" min="0" max="10" class="form-control" id="cgpa" name="cgpa" value="{{ request.form.get('cgpa', '') }}"
required></div>
            <div class="col-md-6"><label for="active_backlogs" class="form-label">Active backlogs</label><input
type="number" min="0" max="40" step="1" class="form-control" id="active_backlogs" name="active_backlogs" value="{{
request.form.get('active_backlogs', '') }}" required></div>
            <div class="col-md-6"><label for="password" class="form-label">Password</label><input type="password"
class="form-control" id="password" name="password" minlength="6" maxlength="128" autocomplete="new-password"
required></div>
            <div class="col-md-6"><label for="confirm_password" class="form-label">Confirm password</label><input
type="password" class="form-control" id="confirm_password" name="confirm_password" minlength="6" maxlength="128"
autocomplete="new-password" required></div>
          </div>
          <button type="submit" class="btn btn-primary mt-4 w-100">Register</button>
        </form>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

A7. Demo data and administrator seed - seed.py

```
from datetime import date, datetime

from werkzeug.security import generate_password_hash

from app import create_app
from models import (
    Application,
    Company,
    PracticeQuestion,
    Resource,
    Student,
    User,
    db,
)


app = create_app()

with app.app_context():
    db.drop_all()
    db.create_all()

    admin = User(
        name="College TPO Admin",
        email="admin@college.edu",
        password_hash=generate_password_hash("admin123"),
        role="admin",
    )
    db.session.add(admin)
    db.session.flush()

    resources = [
        Resource(title="Quantitative Aptitude Basics", category="Aptitude", link_or_text="https://www.indiabix.com/
aptitude/questions-and-answers/", added_by=admin.id),
        Resource(title="Data Interpretation Guide", category="Aptitude", link_or_text="https://www.geeksforgeeks.
org/data-interpretation/", added_by=admin.id),
        Resource(title="Python Coding Practice", category="Coding", link_or_text="https://www.hackerrank.com/
domains/python", added_by=admin.id),
        Resource(title="DSA Interview Questions", category="Coding", link_or_text="https://www.geeksforgeeks.org/
data-structures/", added_by=admin.id),
        Resource(title="SQL Practice Problems", category="Coding", link_or_text="https://leetcode.com/problemset/
database/", added_by=admin.id),
        Resource(title="Common HR Interview Questions", category="Interview", link_or_text="https://www.themuse.
com/advice/25-interview-questions-and-answers", added_by=admin.id),
        Resource(title="STAR Interview Method", category="Interview", link_or_text="Use Situation, Task, Action,
and Result to structure behavioral answers.", added_by=admin.id),
        Resource(title="Resume Writing Checklist", category="Resume", link_or_text="Keep your resume concise,
measurable, truthful, and tailored to the role.", added_by=admin.id),
        Resource(title="ATS-Friendly Resume Tips", category="Resume", link_or_text="Use keywords, standard
headings, and simple formatting for applicant tracking systems.", added_by=admin.id),
        Resource(title="Placement Group Discussion Tips", category="Interview", link_or_text="https://www.
careerizma.com/group-discussion-tips", added_by=admin.id),
    ]
    db.session.add_all(resources)

    student_data = [
        ("Aarav Sharma", "cse2026001@college.edu", "CSE24001", "CSE", 9.2, 0),
        ("Diya Patel", "aiml2026002@college.edu", "AIML24002", "AIML", 8.7, 0),
        ("Rohan Mehta", "it2026003@college.edu", "IT24003", "IT", 8.1, 1),
        ("Ananya Iyer", "ece2026004@college.edu", "ECE24004", "ECE", 7.8, 0),
        ("Kabir Singh", "me2026005@college.edu", "ME24005", "ME", 7.3, 2),
        ("Meera Nair", "cse2026006@college.edu", "CSE24006", "CSE", 8.9, 0),
        ("Arjun Rao", "aiml2026007@college.edu", "AIML24007", "AIML", 7.6, 1),
        ("Sara Khan", "it2026008@college.edu", "IT24008", "IT", 9.0, 0),
    ]

    students = []
    for name, email, roll_no, branch, cgpa, backlogs in student_data:
        user = User(
            name=name,
            email=email,
            password_hash=generate_password_hash("student123"),
            role="student",
        )
        student = Student(
            roll_no=roll_no,
            branch=branch,
            cgpa=cgpa,
            active_backlogs=backlogs,
            resume_filename=None,
        )
        user.student = student
        db.session.add(user)
        students.append(student)

    companies = [
        Company(
            name="TechNova Solutions",
            role_title="Software Engineer",
            package_lpa=12.0,
            allowed_branches="CSE,AIML,IT",
            min_cgpa=8.0,
            max_backlogs=0,
            drive_date=date(2026, 11, 15),
```

```
            status="Open",
        ),
        Company(
            name="DataCore Analytics",
            role_title="Data Analyst",
            package_lpa=8.5,
            allowed_branches="CSE,AIML,IT,ECE",
            min_cgpa=7.5,
            max_backlogs=1,
            drive_date=date(2026, 11, 22),
            status="Screening",
        ),
        Company(
            name="CircuitWorks",
            role_title="Embedded Systems Engineer",
            package_lpa=7.8,
            allowed_branches="ECE,CSE",
            min_cgpa=7.0,
            max_backlogs=2,
            drive_date=date(2026, 12, 1),
            status="Interview",
        ),
        Company(
            name="MechEdge Industries",
            role_title="Graduate Engineer Trainee",
            package_lpa=6.5,
            allowed_branches="ME,ECE",
            min_cgpa=6.5,
            max_backlogs=2,
            drive_date=date(2026, 10, 20),
            status="Closed",
        ),
        Company(
            name="CloudPeak Systems",
            role_title="Cloud Associate",
            package_lpa=10.0,
            allowed_branches="CSE,AIML,IT",
            min_cgpa=7.8,
            max_backlogs=0,
            drive_date=date(2026, 12, 10),
            status="Open",
        ),
    ]
    db.session.add_all(companies)
    db.session.flush()

    question_data = [
        ("Aptitude", "What is 15% of 200?", "20", "25", "30", "35", "C"),
        ("Aptitude", "A train travels 60 km in 1 hour. How far will it travel in 2.5 hours at the same speed?",
"120 km", "150 km", "180 km", "200 km", "B"),
        ("Aptitude", "What is the average of 10, 20, and 30?", "15", "20", "25", "30", "B"),
        ("Aptitude", "If 3x = 18, x equals:", "5", "6", "7", "8", "B"),
        ("Aptitude", "Which number is divisible by 3?", "122", "124", "126", "128", "C"),
        ("Aptitude", "A shop offers a 20% discount on 500 rupees. The discount is:", "80", "100", "120", "150",
"B"),
        ("Aptitude", "What comes next: 2, 4, 8, 16, ?", "24", "30", "32", "34", "C"),
        ("Aptitude", "The ratio 2:3 is equivalent to:", "4:5", "6:9", "8:12", "10:12", "B"),
        ("Coding", "Which data structure uses FIFO?", "Stack", "Queue", "Tree", "Graph", "B"),
        ("Coding", "Which operation has average O(1) complexity in a hash table?", "Lookup", "Sorting",
"Traversal", "Deletion of all entries", "A"),
        ("Coding", "What does SQL SELECT do?", "Updates data", "Deletes data", "Reads data", "Creates tables",
"C"),
        ("Coding", "Which keyword creates a class in Python?", "def", "object", "class", "struct", "C"),
        ("Coding", "A binary search works on:", "Any list", "Sorted lists", "Unsorted trees only", "Hash tables
only", "B"),
        ("Coding", "Which is NOT a stable sorting algorithm by default?", "Merge sort", "Insertion sort", "Bubble
sort", "Quick sort", "D"),
        ("Coding", "What is time complexity of binary search?", "O(1)", "O(log n)", "O(n)", "O(n log n)", "B"),
        ("Coding", "Which SQL clause filters rows?", "GROUP BY", "WHERE", "ORDER BY", "JOIN", "B"),
        ("Interview", "What should you do first when asked a difficult interview question?", "Leave the interview",
 "Ask clarifying questions", "Guess immediately", "Change the topic", "B"),
        ("Interview", "The STAR method is commonly used for:", "Salary negotiation", "Behavioral questions",
"Technical coding tests", "Company research", "B"),
        ("Interview", "Good body language includes:", "Avoiding eye contact", "Active posture and attentive
listening", "Interrupting the interviewer", "Looking only at a phone", "B"),
        ("Interview", "What is the best response to asking about weaknesses?", "Claim you have none", "Give an
honest weakness and improvement plan", "Criticize your team", "Avoid the question", "B"),
        ("Interview", "Why should you research a company before an interview?", "To memorize the logo", "To tailor
answers and ask informed questions", "To skip technical preparation", "To negotiate without criteria", "B"),
        ("Interview", "What is a strong way to close an interview?", "Leave without speaking", "Summarize interest
and ask next steps", "Demand an offer", "Critique the interviewer", "B"),
        ("Interview", "When describing a project, emphasize:", "Only its length", "Your role, actions, and
results", "Only team failures", "Unrelated hobbies", "B"),
        ("Interview", "A thoughtful salary question should be based on:", "Market data and role responsibilities",
"A random number", "The number of sick days", "Another employee's secret pay", "A"),
        ("Resume", "Which resume section highlights degree and institution?", "Education", "Hobbies", "References",
 "Photography", "A"),
        ("Resume", "Strong project bullets should include:", "Buzzwords only", "Actions and measurable results",
"Full source code", "Personal secrets", "B"),
        ("Resume", "An ATS-friendly resume should use:", "Images for all headings", "Simple text and standard
headings", "Multiple columns everywhere", "Unreadable fonts", "B"),
        ("Resume", "A resume should generally be:", "Unlimited in length", "Concise and relevant", "A list of
every course", "Copied from a friend", "B"),
        ("Resume", "Project achievements are strongest when they:", "Use no numbers", "Include measurable impact",
"Mention confidential data", "Are copied", "B"),
        ("Resume", "A resume should be updated when:", "Never", "Skills or achievements change", "Only after
graduation", "An employee leaves", "B"),
    ]
```

```
    # Rebalance the seeded sample so each available Aptitude, Coding, and Interview
    # category contains the 10 questions required by the practice-test engine.
    for index in range(len(question_data) - 6, len(question_data)):
        row = list(question_data[index])
        row[0] = ["Aptitude", "Coding", "Interview"][index % 3]
        question_data[index] = tuple(row)

    db.session.add_all([
        PracticeQuestion(
            category=category,
            question=question,
            option_a=option_a,
            option_b=option_b,
            option_c=option_c,
            option_d=option_d,
            correct_option=correct_option,
        )
        for category, question, option_a, option_b, option_c, option_d, correct_option in question_data
    ])

    applications = [
        Application(student=students[0], company=companies[0], status="Selected", applied_on=datetime(2026, 10, 2,
10, 0), updated_on=datetime(2026, 10, 20, 15, 30)),
        Application(student=students[1], company=companies[0], status="Shortlisted", applied_on=datetime(2026, 10,
3, 9, 15), updated_on=datetime(2026, 10, 21, 11, 0)),
        Application(student=students[2], company=companies[1], status="Interview", applied_on=datetime(2026, 10, 5,
 14, 0), updated_on=datetime(2026, 10, 22, 16, 45)),
        Application(student=students[3], company=companies[2], status="Applied", applied_on=datetime(2026, 10, 8,
12, 30), updated_on=datetime(2026, 10, 8, 12, 30)),
        Application(student=students[4], company=companies[3], status="Rejected", applied_on=datetime(2026, 10, 1,
11, 0), updated_on=datetime(2026, 10, 18, 10, 0)),
        Application(student=students[5], company=companies[4], status="Shortlisted", applied_on=datetime(2026, 10,
9, 10, 20), updated_on=datetime(2026, 10, 23, 9, 30)),
        Application(student=students[7], company=companies[1], status="Selected", applied_on=datetime(2026, 10, 4,
13, 10), updated_on=datetime(2026, 10, 24, 14, 0)),
    ]
    db.session.add_all(applications)
    db.session.commit()

    print("Database reset and seeded successfully.")
```

A8. Role-based decorator - decorators.py

```
"""Backward-compatible decorator import."""

from utils.decorators import role_required

__all__ = ["role_required"]
```